SAT Solver Techniques

A hands-on tutorial: what CaDiCaL, kissat and MiniSat do, with small worked examples

Companion to the ksat ports

Contents

How to read this tutorial 4
Notation 4
The CDCL loop 4
The loop at a glance 4
The running example 5
Unit propagation and the trail 5
Decisions and decision levels 6
Conflicts 6
The implication graph 7
Conflict analysis and the first UIP 7
Recursive clause minimization 8
Shrinking 9
Backjumping 10
On-the-fly strengthening (OTFS) 10
Chronological backtracking 11
Watched literals and blocking literals 12
Search heuristics 13
Choosing the next variable: VSIDS and EVSIDS 13
VMTF: the move-to-front queue 14
Phase saving 14
Target and best phases 15
Rephasing 16
Stable and focused mode 17
Restarts 18
Luby restarts and reluctant doubling 18
Glucose-style EMA restarts 18
Trail reuse on restart 19
Reorder 19
Warmup 20
Random decisions 21
Lucky phases 21
Local search (walk) and how it feeds phases 22
Lookahead and cubing 23
The chapter at a glance 23
Managing learned clauses 24
Why learned clauses must be pruned 24
LBD (glue): a quality score for learned clauses 24
Tiers: keep, keep while used, reducible 25
The reduce step in CaDiCaL 26
Dynamic tiers and reduce in kissat 27
Activity-based reduceDB in MiniSat 28
The used flag and promotion 29
Protecting reason clauses 30
Root-level simplification 30
Garbage collection and arena compaction 31
Variable compaction 32
Simplifying the formula I: removing clauses and variables 32
Preprocessing and inprocessing 33
Resolution, the basic step 33
Subsumption and strengthening 33

Bounded variable elimination 34
Fast elimination (fastel) 35
Gates and definitions 36
Blocked clause elimination 37
Covered clause elimination 37
Globally blocked clauses (conditioning) 38
Instantiation 39
Bounded variable addition (factorization) 39
Model reconstruction: the extension stack 40
Frozen variables 40
Simplifying the formula II: probing, equivalences, strengthening 41
The binary implication graph 42
Failed literal probing 42
Hyper-binary resolution 43
Vivification 44
Equivalent literal substitution 45
Transitive reduction 46
Ternary resolution 46
Backbone computation 47
Congruence closure 47
SAT sweeping with the kitten sub-solver 48
Summary of this chapter 49
Using a solver incrementally 50
Incremental SAT and the IPASIR interface 50
Assumptions are decisions on the first levels 51
The failed-assumption core 52
Adding clauses after a solve 53
Model enumeration with blocking clauses 53
Frozen variables and elimination 54
IPASIR-UP: user propagators 55
Forced phases and the fixed-assignment listener 58
Proofs of unsatisfiability: DRAT and LRAT 59
Overview: who has what 60
The CDCL loop and heuristics (chapters 1–3) 60
Simplifying the formula (chapters 4–5) 61
Incremental use (chapter 6) 62

How to read this tutorial

This tutorial explains the techniques behind modern CDCL SAT solvers (MiniSat, CaDiCaL, kissat), one keyword at a time, each with a tiny formula you can follow by hand. It is deliberately separate from the port documentation (`incremental-ipasir.md`, `shadowing-methodology`): those describe **what the Kotlin ports do and why**, this one teaches **what the techniques are**. Every technique ends with a "where" box: in which upstream file it lives, and whether the ksat Kotlin ports have it.

ksat has four solver ports: MicroSAT (a minimal solver, plain solve only, not covered further here), MiniSat, CaDiCaL and kissat.

The chapters build on each other. Chapter 1 (the CDCL loop) is the foundation; chapters 2–3 make that loop fast; chapters 4–5 shrink the formula itself; chapter 6 is about using a solver incrementally from a program; chapter 7 is a one-page map.

Notation

- **Variables** are lowercase letters $a, b, c, \ldots$. A **literal** is a variable or its negation, written $\overline{a}$ ("not a").
- A **clause** is a disjunction of literals, e.g. $\left(a \vee \overline{b} \vee c\right)$: at least one of its literals must be true. Clauses are named $C_1, C_2, \ldots$.
- A **formula** in CNF is a set (conjunction) of clauses. It is **satisfiable** (SAT) if some assignment makes every clause true, otherwise **unsatisfiable** (UNSAT).
- An **assignment** is written $a = 1$ (true) or $a = 0$ (false). A literal is **falsified** when the assignment makes it false.
- In the solver's **trail** (the ordered list of assignments), $a@2$ means "a was set true at decision level 2"; $\overline{b}@2 \leftarrow C_3$ means "b was set false at level 2 because clause C_3 forced it".
- The same clauses in solver input format (DIMACS) number the variables $a = 1, b = 2, \ldots$ and write a negation as a minus sign: $\left(a \vee \overline{b} \vee c\right)$ is `1 -2 3 0`.

The CDCL loop

Every solver in this tutorial (MiniSat, CaDiCaL, kissat, and the ksat ports) runs the same basic algorithm: **conflict-driven clause learning** (CDCL). It guesses a value for a variable, follows the consequences, and when the guesses lead to a contradiction it works out *why*, writes the reason down as a new clause, and jumps back. Everything in later chapters (heuristics, restarts, clause deletion, simplification) is built around this loop. This chapter walks through it with one small formula, from the first decision to the final model.

The loop at a glance

Here is the whole algorithm. Each line is explained in its own section below.

```
level = 0
loop:
  conflict = propagate()             // unit propagation: extend the trail
  if conflict != none:
    if level == 0: return UNSAT      // contradiction without any guess
    learned, jump = analyze(conflict) // 1-UIP clause and its backjump level
    minimize(learned)                // drop redundant literals (+ shrink)
    backtrack(jump)                  // undo every level above 'jump'
    add(learned)                     // it is unit now: assign its UIP literal
  else:
    if every variable is assigned: return SAT
    maybe restart / reduce / simplify   // chapters 2 to 5
    level = level + 1
    decide()                         // pick an unassigned variable and a value
```

Three data structures carry the state: the *trail* (the ordered list of assignments, each with its decision level and its reason clause), the *clause database* (original plus learned clauses) and the *watch lists* that make `propagate()` fast (last section of this chapter).

CaDiCaL: `internal.cpp` (`Internal::cdcl_loop_with_inprocessing`) · kissat: `search.c` (`kissat_search`) · MiniSat: `Solver.cc` (`Solver::search`) · ksat port: ported (all three cores)

The running example

We use the following eight clauses over the variables $a, b, \ldots, k$ for the whole chapter.

Name	Clause	DIMACS
C_1	$(\overline{a} \vee b)$	`-1 2 0`
C_2	$(\overline{c} \vee d)$	`-3 4 0`
C_3	$(\overline{c} \vee e)$	`-3 5 0`
C_4	$\left(\overline{f} \vee g\right)$	`-6 7 0`
C_5	$\left(\overline{h} \vee i\right)$	`-8 9 0`
C_6	$\left(\overline{i} \vee j\right)$	`-9 10 0`
C_7	$\left(\overline{i} \vee \overline{a} \vee \overline{d} \vee k\right)$	`-9 -1 -4 11 0`
C_8	$\left(\overline{j} \vee \overline{k} \vee \overline{b} \vee \overline{e}\right)$	`-10 -11 -2 -5 0`

The formula is satisfiable, but not every guess works: C_8 forbids that j, k, b and e are all true, and the binary clauses make it easy to force exactly that. To keep the walk reproducible we use a deliberately dumb decision rule: *pick the alphabetically first unassigned variable and set it to 1*. Real solvers choose much more cleverly (chapter 2), but the loop itself does not care how the choice is made.

Unit propagation and the trail

A clause is **unit** when all of its literals but one are false and the remaining one is unassigned. Then that last literal must be true, otherwise the clause would be violated. Unit propagation applies this rule over and over until nothing changes. It is the only kind of reasoning the solver does between guesses, and it is where a CDCL solver spends most of its time.

Every assignment goes onto the **trail** in the order it was made, together with two facts: the decision level at which it happened and the clause that forced it (its *reason*). A guessed assignment has no reason. The trail is both the current partial assignment and the record of why each value is there, which conflict analysis needs later.

> **Propagating after the first guess**
> Nothing is unit at the start (no clause has only one literal), so the solver has to guess. It sets $a = 1$ at level 1. Now $C_1 = (\overline{a} \vee b)$ has its first literal false and b unassigned, so it is unit and forces $b = 1$. No other clause becomes unit. The trail is
>
> $$a@1, \quad b@1 \leftarrow C_1$$
>
> C_7 also contains $\overline{a}$, which is now false, but it still has three unassigned literals, so it says nothing yet.

Why it matters: propagation is cheap logical inference. One guess can fix dozens of variables without any search. The cost is that it must be done after every single assignment, which is why solvers spend so much engineering on the watch lists (see the last section).

CaDiCaL: `propagate.cpp` · kissat: `propsearch.c`, `proplit.h` · MiniSat: `Solver.cc` (`propagate`, `uncheckedEnqueue`) · ksat port: ported

Decisions and decision levels

When propagation stops without a conflict and some variables are still open, the solver makes a *decision*: it picks an unassigned variable and a value. Each decision opens a new **decision level**. Level 0 holds only facts that follow from the formula alone (no guess involved); level n holds the n-th decision plus everything propagated from it. The level is what lets the solver undo exactly one guess and its consequences: drop everything above a given level.

Four decisions

Applying our rule (first unassigned variable, value 1) and propagating after each step:

Level	Decision	Trail after propagation (new part)
1	$a = 1$	$a@1, \quad b@1 \leftarrow C_1$
2	$c = 1$	$c@2, \quad d@2 \leftarrow C_2, \quad e@2 \leftarrow C_3$
3	$f = 1$	$f@3, \quad g@3 \leftarrow C_4$
4	$h = 1$	$h@4, \quad i@4 \leftarrow C_5, \quad j@4 \leftarrow C_6, \quad k@4 \leftarrow C_7$

At level 4, h forces i through C_5. Then i forces j through C_6, and $C_7 = \left(\overline{i} \vee \overline{a} \vee \overline{d} \vee k\right)$ forces k, because i, a and d are all true by now. The order matters here: the solver handles all clauses watching i first (C_6 gives j, C_7 gives k); only then does it process j and find C_8 false (next section). Level 3 (f, g) has nothing to do with the rest; keep an eye on it, it matters for backjumping.

Why it matters: decisions are the only place where the solver searches. A good choice of variable and value (chapter 2) can shorten the run by orders of magnitude. The decision level costs one integer per variable and one "where does this level start on the trail" entry per level.

CaDiCaL: `decide.cpp` · kissat: `decide.c` · MiniSat: `Solver.cc` (`pickBranchLit`, `newDecisionLevel`) · ksat port: ported

Conflicts

A **conflict** happens when propagation finds a clause whose literals are all false. Then the current assignment cannot be extended to a model, and at least one of the decisions was wrong. If the conflict happens at level 0, no decision is involved and the formula is unsatisfiable. Otherwise the solver analyzes it.

The conflict at level 4

After the clauses watching i are done, the trail contains $j@4$, $k@4$, $b@1$ and $e@2$. The solver now processes j and looks at $C_8 = \left(\overline{j} \vee \overline{k} \vee \overline{b} \vee \overline{e}\right)$: all four literals are false. C_8 is the **conflict clause**. Two of its literals belong to level 4 (j, k), one to level 1 (b) and one to level 2 (e).

A naive solver would now flip the last decision ($h = 0$) and continue. That is plain backtracking (DPLL). It is correct, but it forgets everything it just found out and happily repeats the same mistake in another part of the search tree. CDCL instead asks which assignments *together* caused the conflict.

CaDiCaL: `propagate.cpp` (sets `conflict`) · kissat: `proplit.h` (returns the conflict clause) · MiniSat: `Solver.cc` (`propagate` returns `confl`) · ksat port: ported

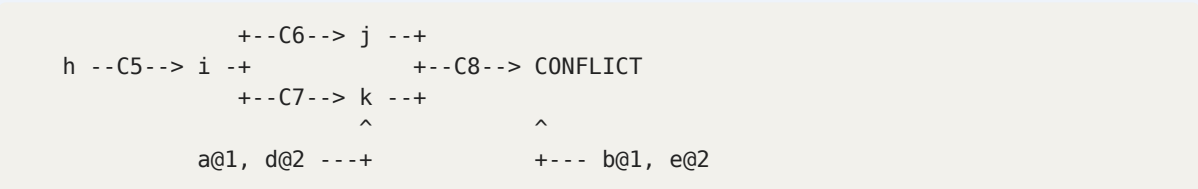
The implication graph

The reasons on the trail form a directed graph. Each assigned literal is a node. For a propagated literal x with reason clause C, there is an edge from every *other* variable of C to x: those assignments are what made C unit. Decisions have no incoming edges. The conflict gets an extra node, called κ here, with edges from every literal of the conflict clause. The solver never builds this graph explicitly; the reason pointers on the trail are the graph.

The graph of our conflict

Node	Level	Reason	Incoming edges from
a	1	decision	none
b	1	C_1	a
c	2	decision	none
d	2	C_2	c
e	2	C_3	c
f	3	decision	none
g	3	C_4	f
h	4	decision	none
i	4	C_5	h
j	4	C_6	i
k	4	C_7	i, a, d
κ		C_8	j, k, b, e

The level-4 part, drawn out:



Nodes f and g (level 3) have no path to κ: they played no part in the conflict.

Why it matters: any set of nodes that separates the decisions from κ (a **cut**) is a reason for the conflict, and the negation of a cut is a clause the formula implies. Conflict analysis is the art of picking a good cut.

CaDiCaL: `var.hpp` (`Var::reason`, `Var::level`) · kissat: `assign.h` (`assigned`, `reason`) · MiniSat: `Solver.h` (`VarData { reason, level }`) · ksat port: ported

Conflict analysis and the first UIP

The solver walks backwards from the conflict clause, using **resolution**: if a clause contains $\overline{x}$ and the reason of x contains x, then the two clauses together imply the clause with all their other literals. Resolving the current clause with the reason of one of its literals replaces that literal by “the things that caused it”. This is the same as moving the cut one step towards the decisions.

The walk handles only literals of the conflict level (level 4 here), always the one assigned last on the trail, and stops as soon as exactly one conflict-level literal is left. That literal is the **first unique implication point** (1-UIP): a node on the conflict level that every path from the level’s decision to κ passes through,

and the one closest to the conflict. The decision itself is always a UIP (the last resort), but usually there is one earlier.

Deriving the 1-UIP clause

Start from $C_8 = \left(\overline{j} \vee \overline{k} \vee \overline{b} \vee \overline{e}\right)$. Level-4 literals: $\overline{j}$, $\overline{k}$, so two are open.

Step	Resolve on	Resulting clause	Level-4 lits
start		$\left(\overline{j} \vee \overline{k} \vee \overline{b} \vee \overline{e}\right)$	j, k
1	k with C_7	$\left(\overline{j} \vee \overline{b} \vee \overline{e} \vee \overline{i} \vee \overline{a} \vee \overline{d}\right)$	j, i
2	j with C_6	$\left(\overline{i} \vee \overline{a} \vee \overline{b} \vee \overline{d} \vee \overline{e}\right)$	i

Step 1 takes k first because it was assigned after j. After step 2 only $\overline{i}$ is left on level 4, so i is the first UIP. The learned clause is

$$L_1 = \left(\overline{i} \vee \overline{a} \vee \overline{b} \vee \overline{d} \vee \overline{e}\right)$$

Read it as: "i, a, b, d and e cannot all be true". In the graph, $\{i, a, b, d, e\}$ is the cut: every path into κ passes through one of them. Other cuts give other implied clauses. The decision h is also a UIP of level 4; cutting there (one more resolution, with C_5) gives $\left(\overline{h} \vee \overline{a} \vee \overline{b} \vee \overline{d} \vee \overline{e}\right)$. Cutting right behind all decisions gives the "decision clause" $\left(\overline{h} \vee \overline{a} \vee \overline{c}\right)$, which talks only about the guesses rather than about what went wrong.

Why it helps: the learned clause is implied by the formula (each step is a resolution), so adding it never removes a model. It blocks not just this exact combination of guesses but every future assignment that makes i, a, b, d, e true, wherever in the search it appears. Having exactly one literal from the conflict level is what makes the clause **asserting**: after the jump back it is unit (section "Backjumping" below). Among the possible cuts, 1-UIP has been the empirical winner since Chaff (2001). Cost: one backwards pass over the conflict level of the trail, with a "seen" flag per variable.

CaDiCaL: `analyze.cpp` (`Internal::analyze`, `analyze_reason`) · kissat: `analyze.c`, `deduce.c` (`kissat_deduce_first_uip_clause`), `learn.c` · MiniSat: `Solver.cc` (`Solver::analyze`) · ksat port: ported

Recursive clause minimization

The 1-UIP clause often contains literals that are implied by other literals of the same clause. Such a literal is **redundant**: removing it still gives a clause the formula implies. The test is recursive. A false literal $\overline{x}$ in the learned clause can be dropped if x was propagated (not decided) and every other literal of x's reason is either already in the learned clause, assigned at level 0, or itself redundant by the same test. Decisions are never redundant, because they have no reason to look into.

Removing $\overline{x}$ this way is just more resolution: resolve the learned clause with the reason of x. Since all the reason's other literals are already in the clause, nothing new comes in and $\overline{x}$ disappears.

Removing the redundant literal from the learned clause

$L_1 = \left(\overline{i} \vee \overline{a} \vee \overline{b} \vee \overline{d} \vee \overline{e}\right)$. Check every literal except the UIP literal $\overline{i}$:

- $\overline{a}$: a is a decision. Keep.
- $\overline{b}$: b was forced by $C_1 = (\overline{a} \vee b)$. Its only other literal is $\overline{a}$, which is in L_1. Redundant, remove.
- $\overline{d}$: forced by $C_2 = (\overline{c} \vee d)$. $\overline{c}$ is not in L_1 and c is a decision. Keep.
- $\overline{e}$: forced by $C_3 = (\overline{c} \vee e)$. Same as d. Keep.

Result: $L_2 = \left(\overline{i} \vee \overline{a} \vee \overline{d} \vee \overline{e}\right)$. It is the resolvent of L_1 and C_1 on b. Intuitively: if a is true then b is true anyway, so mentioning b adds nothing.

Here the recursion stops after one step. In general the check follows reasons several levels deep, which is why it is "recursive" (MiniSat's `ccmin_mode = 2`, also called deep minimization; mode 1 only looks one step). MiniSat speeds it up with an *abstract level* bitmask: if a reason contains a literal from a decision level that does not occur in the learned clause at all, the literal cannot be redundant and the search stops early. CaDiCaL and kissat cache the outcome per variable with "poison" (known not removable) and "removable" flags, and cap the depth (`minimizedepth`, default 1000).

Why it helps: shorter clauses propagate earlier, take less memory and make later conflict analysis cheaper. Cost: a depth-first search over reasons, bounded by the caching.

CaDiCaL: `minimize.cpp` (`minimize_literal`, `minimize_clause`) · kissat: `minimize.c` · MiniSat: `Solver.cc` (`litRedundant`, called from `analyze` with `ccmin_mode = 2`) · ksat port: ported

Shrinking

Minimization only deletes literals. **Shrinking** (Fleury and Biere) goes further: it looks at the learned clause one decision level at a time, and if a level contributes two or more literals, it tries to replace all of them by a single literal from that level. It does the same backwards walk as 1-UIP analysis, but restricted to one level: resolve away the latest literal of that level until only one is left. That literal is a UIP of the block, and its negation replaces the block. If the walk reaches a literal from a *lower* level that is not already in the clause and not removable by minimization, shrinking fails for this block and leaves it as it was.

Shrinking the level-2 block

Sort the literals of $L_2 = \left(\overline{i} \vee \overline{a} \vee \overline{d} \vee \overline{e}\right)$ by level: level 4: $\overline{i}$ (the UIP), level 2: $\overline{d}$, $\overline{e}$, level 1: $\overline{a}$. Only level 2 has more than one literal.

Walk back on level 2: e came from $C_3 = (\overline{c} \vee e)$, giving c; d came from $C_2 = (\overline{c} \vee d)$, giving c again. Only c is left, so c is the UIP of level 2, and the block $\left\{\overline{d}, \overline{e}\right\}$ is replaced by $\overline{c}$:

$$L_3 = \left(\overline{i} \vee \overline{a} \vee \overline{c}\right)$$

Shrinking the unminimized L_1 gives the same result: its level-1 block $\left\{\overline{a}, \overline{b}\right\}$ shrinks to $\overline{a}$ (b was forced by a).

Shrinking never changes which levels occur in the clause, because each block is replaced by a literal of its own level. So L_1, L_2 and L_3 all have literals on levels 4, 2 and 1. This matters for the jump back in the next section.

Why it helps: shorter clauses again, and literals that sit closer to the decisions, which tend to be more general. Upstream it runs as one pass together with minimization (CaDiCaL `shrink_and_minimize_clause`). Cost: one extra walk per level block, bounded in CaDiCaL by a priority queue over trail positions (`shrinkreap`). Both CaDiCaL and kissat default to the full variant (`shrink = 3`), which also calls minimization for lower-level literals met during the walk.

CaDiCaL: `shrink.cpp` (option `shrink`, default 3) · kissat: `shrink.c` (option `shrink`, default 3) · MiniSat: — · ksat port: not ported (the ports stop after recursive minimization)

Backjumping

Now the solver undoes assignments. It does not just go back one level. It jumps to the second-highest level among the learned clause's literals, the **backjump level**. At that level all literals of the learned clause are false except the UIP literal, which is unassigned. So the learned clause is unit and immediately forces the opposite of the UIP. The solver jumps as far back as possible while keeping that property.

> **Jumping from level 4 to level 2**
> Take the minimized clause $L_2 = \left(\overline{i} \vee \overline{a} \vee \overline{d} \vee \overline{e}\right)$. Its literals live on these levels: $\overline{i}$ on 4, $\overline{d}$ and $\overline{e}$ on 2, $\overline{a}$ on 1. The second-highest level is 2, so the solver removes everything above level 2: f, g (level 3) and h, i, j, k (level 4). The trail is
>
> $$a@1, \quad b@1 \leftarrow C_1, \quad c@2, \quad d@2 \leftarrow C_2, \quad e@2 \leftarrow C_3$$
>
> In L_2, $\overline{a}$, $\overline{d}$, $\overline{e}$ are false and i is unassigned, so L_2 is unit and forces $\overline{i}@2$. Then $C_5 = \left(\overline{h} \vee i\right)$ is unit too and forces $\overline{h}@2$:
>
> $$\ldots, \quad e@2 \leftarrow C_3, \quad \overline{i}@2 \leftarrow L_2, \quad \overline{h}@2 \leftarrow C_5$$
>
> Which clause is actually added depends on the solver. The ksat ports minimize but do not shrink, so they add L_2. CaDiCaL and kissat shrink by default and would add $L_3 = \left(\overline{i} \vee \overline{a} \vee \overline{c}\right)$ instead (then $\overline{i}@2 \leftarrow L_3$). The backjump level is 2 in all cases, also for the unminimized L_1, and the forced literal is the same.
>
> The solver continues. It decides $f = 1$ (level 3, forcing g), then $j = 1$ (level 4). Now C_8 has $\overline{j}$, $\overline{b}$, $\overline{e}$ false and forces $\overline{k}@4 \leftarrow C_8$. All variables are assigned and no clause is false: the model is $a = b = c = d = e = f = g = j = 1$, $h = i = k = 0$.

Why it helps: level 3 was not involved in the conflict. If every choice at level 4 failed, plain backtracking would next flip f at level 3, although f had nothing to do with the conflict; backjumping skips that. Note the price: the work spent on level 3 (f, g) is thrown away and has to be redone. In this example it is two assignments; in real instances it can be thousands, which is what chronological backtracking (below) addresses.

CaDiCaL: `backtrack.cpp` (`Internal::backtrack`), `analyze.cpp` (`find_conflict_level`, `determine_actual_backtrack_level`) · kissat: `backtrack.c`, `learn.c` · MiniSat: `Solver.cc` (`cancelUntil`; the backjump level comes out of `analyze`) · ksat port: ported

On-the-fly strengthening (OTFS)

During conflict analysis the solver resolves the current clause with reason clauses. Sometimes the result is exactly the reason clause minus the literal just resolved on. Then the reason clause itself can be made shorter: the formula implies the reason without that literal, so the solver deletes the literal from the reason clause in place. This is **on-the-fly self-subsumption**, or on-the-fly strengthening. The test is cheap because analysis already counts sizes: if the resolvent is strictly smaller than the reason (counting only literals not fixed at level 0), it must be the reason minus the pivot. The strengthened clause has all literals false, so it becomes the new conflict clause and analysis restarts from it. If it has only one literal on the conflict level, it is already asserting: the solver backjumps and uses it as the reason directly, without adding any new clause.

Our running example does not trigger it (step 1 resolves with C_7, giving a resolvent of six literals against a reason of four; step 2 uses the binary C_6, and the test requires more than two literals). So here is a separate three-clause formula.

Strengthening a reason clause (separate toy formula)
$D_1 = \left(\overline{b} \vee c\right)$, $D_2 = (\overline{a} \vee \overline{c} \vee d)$, $D_3 = \left(\overline{c} \vee \overline{d}\right)$.

Decide $a@1$; nothing propagates. Decide $b@2$: D_1 forces $c@2$. Both D_2 and D_3 watch $\overline{c}$; assume the solver visits D_2 before D_3. Then D_2 forces $d@2$, and D_3 is false. Conflict. (The order matters: visiting D_3 first would force $\overline{d}@2 \leftarrow D_3$ and make D_2 the conflict. The reason resolved with would then be the binary D_3, and the test needs more than two literals, so no OTFS.)

Resolve D_3 with the reason of d, which is D_2, on d: the result is $(\overline{c} \vee \overline{a})$, two literals, while D_2 has three. So $(\overline{a} \vee \overline{c})$ is exactly D_2 minus d, and D_2 is strengthened in place to $(\overline{a} \vee \overline{c})$. It has one level-2 literal ($\overline{c}$), so the solver jumps back to level 1 and assigns $\overline{c}@1 \leftarrow D_2$.

Plain 1-UIP would learn the same $(\overline{a} \vee \overline{c})$ as a *new* clause and keep the old D_2, which is now subsumed (a superset of a known clause) and only wastes memory.

Why it helps: it shrinks original and learned clauses for free during analysis and avoids storing redundant copies. Cost: the strengthened clause has to be re-watched (its watched literal may be the one removed), and with proof output (a DRAT/LRAT certificate, chapter 6) the change must be logged. If this happens on the first resolution step (CaDiCaL `analyze.cpp`: `resolved == 1`, the same test in kissat's `deduce.c`) and the strengthened clause is also smaller than the conflict clause, both clauses come straight from the clause database and the strengthened one subsumes the conflict clause, so CaDiCaL and kissat delete the conflict clause.

CaDiCaL: `analyze.cpp` (`on_the_fly_strengthen`, `otfs_strengthen_clause`; option `otfs`, default 1) · kissat: `deduce.c` (the test), `strengthen.c` (`kissat_on_the_fly_strengthen`, `kissat_on_the_fly_subsume`; option `otfs`) · MiniSat: — · ksat port: not ported

Chronological backtracking

Backjumping throws away all levels above the backjump level, even the ones that had nothing to do with the conflict. On large instances that can be thousands of assignments, which the solver will often just recreate. **Chronological backtracking** (Nadel and Ryvchin) undoes only the *last* level and keeps the rest. The learned clause is still unit, but the forced literal now gets the level it would have had after a backjump (the highest level of the clause's other literals), even though it sits higher up on the trail.

Keeping level 3
Instead of jumping to level 2, the solver backtracks only to level 3. The learned clause L_2 is unit as before and forces $\overline{i}$, then C_5 forces $\overline{h}$. Both get level 2, because all other literals of their reasons are on level 2 or below:

$$a@1, \quad b@1, \quad c@2, \quad d@2, \quad e@2, \quad f@3, \quad g@3, \quad \overline{i}@2 \leftarrow L_2, \quad \overline{h}@2 \leftarrow C_5$$

f and g survive. But the trail is no longer sorted by level: two level-2 literals come after level-3 ones. If the solver later backtracks to level 2, it must keep $\overline{i}$ and $\overline{h}$ and remove f, g from the middle of the trail.

When is it used? Both CaDiCaL and kissat backjump by default and fall back to chronological backtracking when the jump would skip more than 100 levels (CaDiCaL `chronolevelim`, kissat `chronolevels`). CaDiCaL additionally has a trail-reuse rule (`chronoreusetrail`, on by default): it keeps the levels up to the one that holds the best-scored variable among those it would undo, capped at the last level. In our example that cap is level 3, so CaDiCaL 2.1.3 with default options would actually keep level 3 here, while kissat (skipping only one level, well below 100) jumps to level 2. MiniSat always backjumps.

Why it helps / cost: it saves re-propagating the skipped levels, which pays off on instances with very deep trails. The price is complexity everywhere: propagation must compute the real level of each forced literal, conflict analysis must first find the actual conflict level (it may be below the current level), and backtracking must handle out-of-order literals.

CaDiCaL: `analyze.cpp` (`determine_actual_backtrack_level`, `find_conflict_level`), `propagate.cpp` (`assignment_level`), `backtrack.cpp` (options `chrono`, `chronolevelim`, `chronoreusetrail`) · kissat: `learn.c` (`kissat_determine_new_level`; options `chrono`, `chronolevels`) · MiniSat: — · ksat port: not ported (the ports always backjump)

Watched literals and blocking literals

Unit propagation has to notice the moment a clause becomes unit. Checking every clause after every assignment would be far too slow. The **two-watched-literal** scheme (Chaff, 2001) uses one observation: a clause can only become unit or false when one of its last two non-false literals becomes false. So the solver watches two non-false literals per clause. For each literal x it keeps a watch list of the clauses that watch $\overline{x}$. When x becomes true it visits only those clauses. For each visited clause:

1. If the other watched literal is true, the clause is satisfied; nothing to do.
2. Otherwise look for another literal that is not false. If there is one, watch it instead (move the clause to that literal's watch list).
3. If there is none, the clause is unit (the other watched literal is forced) or, if the other watched literal is false too, a conflict.

A nice property: on backtracking nothing has to be updated. Undoing assignments only turns false literals back into unassigned ones, so two watched literals that were fine stay fine.

A **blocking literal** (Chu, Harwood and Stuckey) is a further shortcut. Each watch-list entry stores, next to the clause reference, one other literal of the clause. If that literal is true, the clause is satisfied and the solver skips it without loading the clause from memory at all. Since memory access is the real cost of propagation, this saves a lot.

Watches on clause C7 during the run

Assume the solver watches the first two literals of $C_7 = \left(\overline{i} \vee \overline{a} \vee \overline{d} \vee k\right)$ as written, i.e. $\overline{i}$ and $\overline{a}$ (MiniSat actually sorts a clause's literals when adding it; the idea is the same).

Assignment	Watched after	What happens to C_7
$a@1$	$\overline{i}$, $\overline{d}$	$\overline{a}$ became false. $\overline{d}$ is unassigned, so the watch moves there.
$b@1$, $c@2$	unchanged	Not in the watch lists of b or c: not visited.
$d@2$	$\overline{i}$, k	$\overline{d}$ became false. $\overline{a}$ is false, k is unassigned: the watch moves to k.
e, f, g, h	unchanged	Not visited.
$i@4$	$\overline{i}$, k	$\overline{i}$ became false. No replacement ($\overline{a}$, $\overline{d}$ false). The other watch k is unassigned: unit, $k@4 \leftarrow C_7$.

Over nine assignments C_7 was opened three times. When i became true, the watch entry's blocking literal was $\overline{a}$ (the other watch at the time the entry was created), which is false, so the clause had to be loaded. MiniSat then stores k as the new blocking literal of that entry, so a later visit while k is true skips C_7 without touching it.

Why it helps: propagation cost depends on the number of clause visits, not the number of clauses. An assignment only touches the one watch list of the literal that became false. Cost: two watch entries per clause (plus the blocking literal), and the clause's literal order changes as watches move. Binary

clauses are special: both literals are always watched, and kissat stores them only in the watch lists, with no separate clause in memory.

CaDiCaL: `propagate.cpp`, `watch.hpp` (`Watch { clause, blit, size }`) · kissat: `proplit.h`, `watch.h` (binary clauses are watch-only) · MiniSat: `Solver.cc` (`propagate`), `Solver.h` (`Watcher { cref, blocker }`) · ksat port: ported, including blocking literals (the CaDiCaL port also has the saved-position replacement search)

Search heuristics

Chapter 1 left two questions open. When the solver has to make a decision, *which variable* does it pick, and *which value* does it give it? And when the search seems stuck, when should it give up on the current trail and start over? The CDCL loop is correct for any answer, but its speed depends almost entirely on these choices. This chapter goes through the heuristics that answer them, from the classic ones every solver has (VSIDS, phase saving, restarts) to the extra machinery in CaDiCaL and kissat (two search modes, rephasing, local search).

Two words are used throughout. A variable is *bumped* when it takes part in conflict analysis (it occurs in the learned clause or was resolved away while deriving it). The *glue* (also called LBD) of a learned clause is the number of different decision levels among its literals; small glue means "this clause connects few decisions", which is a sign of a useful clause.

A short preview of one more idea, because many sections below depend on it. CaDiCaL and kissat switch back and forth between two *search modes*. In *focused* mode they restart very often and follow the latest conflicts closely (good for proving UNSAT). In *stable* mode they restart rarely and change direction slowly (good for finding models). Several heuristics below are used in only one of the two modes; the section "Stable and focused mode" explains the modes in full. MiniSat has only one mode.

Choosing the next variable: VSIDS and EVSIDS

Intuition. Variables that showed up in recent conflicts are probably the ones that matter right now. So every variable gets an activity score; conflict analysis adds to the score of each bumped variable, and the next decision takes the unassigned variable with the highest score. To make "recent" count more than "long ago", old score contributions must fade.

The original VSIDS (Chaff) divided all scores periodically. MiniSat introduced the trick that everyone uses today, called EVSIDS (exponential VSIDS): instead of shrinking every score after each conflict, grow the *increment*. After every conflict the increment is multiplied by $\frac{1}{0.95} \approx 1.0526$. A bump from the current conflict is then worth $\frac{1}{0.95}$ times a bump from the previous conflict, which is the same ranking as "multiply all scores by 0.95, then add 1", but costs one multiplication instead of touching every variable.

EVSIDS scores over three conflicts

Four variables a, b, c, d start with score 0, increment 1, factor $f = \frac{1}{0.95}$.

conflict	bumped	scores after bumping	new increment
1	a, b	$a = 1, b = 1$	1.0526
2	b, c	$b = 2.0526, c = 1.0526$	1.1080
3	a, c	$a = 2.1080, c = 2.1607, b = 2.0526$	1.1664

Final ranking: $c > a > b > d$. Both a and c were bumped twice, but c's bumps are more recent, so it wins. b was also bumped twice and loses to both because its last bump was in conflict 2. The "decay everything" view gives $a = 1.9025$, $b = 1.8525$, $c = 1.95$, which is exactly the table divided by f^2: same order, as promised.

The scores grow without bound, so solvers rescale before they overflow a `double`: MiniSat multiplies all activities (and the increment) by 10^{-100} when one exceeds 10^{100}; CaDiCaL and kissat use the limit 10^{150} and divide everything by the largest score (or by the increment, if that is larger).

The unassigned variable with the highest score is found with a binary *max-heap* keyed on the score. Bumping a variable moves it up in the heap. Assigned variables are not removed eagerly: when the top of the heap is already assigned it is popped and the next one is tried, and unassigned variables are pushed back into the heap when backtracking unassigns them.

Why it helps / cost. VSIDS concentrates the search on the part of the formula that currently produces conflicts, so learned clauses build on each other. The cost is $O(\log n)$ per bump and per pop, which is noticeable when conflicts involve hundreds of variables; that is one reason for VMTF below.

CaDiCaL: `analyze.cpp` (`bump_variable_score`, `scorefactor` = 950 per mille), `score.cpp`, `heap.hpp`; EVSIDS is used only in stable mode · kissat: `bump.c` (`decay` = 50 per mille, `MAX_SCORE` = 1e150), `heap.c`, `decide.c`; stable mode only · MiniSat: `Solver.h` (`varBumpActivity`, `varDecayActivity`), `Solver.cc` (`var-decay` = 0.95), used always · ksat port: ported in all three (reason-side bumping, `bumpreason`/`bumpreasons`, is not)

VMTF: the move-to-front queue

Intuition. VSIDS keeps a precise score; VMTF (variable move-to-front) keeps only an order. All variables sit in a doubly linked list. When a variable is bumped it is moved to the front. The next decision is the first unassigned variable from the front. There is no decay because there is no score: whoever was bumped most recently is first, period.

Each variable carries a time stamp (`btab` in CaDiCaL, `stamp` in kissat) that increases with every move to the front, so "in front of" is just "has a larger stamp". When one conflict bumps several variables, both solvers first sort them by their *old* stamp and then move them in that order, so their relative order is kept.

> **Bumping two variables in a VMTF queue**
>
> Queue from front to back, with stamps: $a(5), b(4), c(3), d(2), e(1)$.
>
> A conflict bumps b and d. Sorted by old stamp: $d(2)$ first, then $b(4)$. Move d to the front (stamp 6), then b (stamp 7). New queue: $b(7), d(6), a(5), c(3), e(1)$.
>
> b was in front of d before and still is. If a and b are assigned, the next decision is d.

To avoid scanning assigned variables at the front over and over, the queue keeps a pointer to the last place where an unassigned variable was found (`queue.unassigned` in CaDiCaL, `queue.search` in kissat). When backtracking unassigns a variable with a larger stamp than the pointer's variable, the pointer moves forward to it.

Why it helps / cost. A bump is $O(1)$ list surgery instead of a heap update, and the search follows recent conflicts very aggressively. That aggressiveness suits "focused" search with frequent restarts; the smoother VSIDS ranking suits long "stable" runs. This is why CaDiCaL and kissat use both, one per mode (see stable and focused mode below).

CaDiCaL: `queue.hpp`, `queue.cpp`, `analyze.cpp` (`bump_queue`), `decide.cpp` (`next_decision_variable_on_queue`); focused mode only · kissat: `queue.c`, `inlinequeue.h`, `bump.c` (`move_analyzed_variables_to_front_of_queue`), `decide.c`; focused mode only · MiniSat: — · ksat port: ported (CaDiCaL and kissat)

Phase saving

Intuition. VSIDS picks the variable; something must pick the value. Backjumping often throws away large parts of the trail that were perfectly fine, only because they sat above the level the solver jumps

back to. Phase saving remembers, for each variable, the value it had the last time it was assigned, and the next decision on that variable uses that value again. The solver then quickly rebuilds the good parts of the old assignment instead of exploring the opposite value.

> **Rebuilding a trail with saved phases**
> Clauses $C_1 = \left(a \vee \overline{b}\right)$, $C_2 = (b \vee c)$, plus others not shown. The trail is
>
> $$\overline{a}@1, \quad \overline{b}@1 \leftarrow C_1, \quad c@1 \leftarrow C_2, \quad \ldots$$
>
> Later a conflict at level 5 produces a learned clause that sends the solver back to level 0, erasing all of this. The saved phases are now $a = 0, b = 0, c = 1$.
>
> When a is decided again, it gets value 0 (saved), and propagation rebuilds $\overline{b}$ and c exactly as before. Without phase saving, CaDiCaL and kissat would give a their default value, true (option `phase` = 1). With $a = 1$, C_1 is satisfied and nothing is forced, so the solver would wander into a different part of the search space. MiniSat's default value is false (it starts every variable with `polarity` = true, which in MiniSat means "use the negative literal"), so for MiniSat this particular decision would come out the same; with a saved value of 1 it would be the other way round.

When the phase is saved differs slightly: MiniSat saves it when a variable is unassigned (`cancelUntil`, full phase saving `phase-saving` = 2 by default), CaDiCaL and kissat save it at assignment time.

Why it helps / cost. Restarts and long backjumps become cheap, because the solver does not lose the partial solution it had built. It costs one byte per variable and one store per assignment. The risk is getting stuck: if the saved assignment is in a bad region, phase saving keeps steering back into it. Rephasing (below) is the remedy.

CaDiCaL: `propagate.cpp` (`phases.saved[idx] = tmp`), `decide.cpp` (`decide_phase`) · kissat: `inlineassign.h` (`SAVED (idx)`), `decide.c` (`kissat_decide_phase`), option `phasesaving` · MiniSat: `Solver.cc` (`cancelUntil`, `pickBranchLit`) · ksat port: ported

Target and best phases

Intuition. Phase saving remembers the *last* value of every variable, including values from a trail that was about to run into a conflict. A better memory is: "the assignment from the longest trail that propagated without a conflict". That trail is the closest the solver has come to a full model. CaDiCaL and kissat keep two such snapshots:

- the *target* phases: a copy of the saved phases taken whenever the conflict-free trail is longer than the current target height. The target height is reset to 0 at every rephase, so the target follows recent progress;
- the *best* phases: the same, but compared against the longest conflict-free trail ever seen (reset only when a rephase installs the best phases).

The snapshot is taken on backtrack. After a conflict, the trail of the conflicting level is not conflict-free, so CaDiCaL measures up to `no_conflict_until` (the trail position reached by the last conflict-free propagation), and kissat first undoes the conflict level and then measures.

In stable mode, decisions use the target phase if one is set, otherwise the saved phase. The best phases are not used directly; rephasing copies them into the saved phases.

> **Target and best heights**
> A formula with 10 variables, in stable mode. Successive backtracks see conflict-free trails of length 4, 7, 5, 6:

conflict-free trail	target height	best height	effect
4	4	4	copy to target and best
7	7	7	copy to target and best
(rephase)	0	7	target height reset
5	5	7	copy to target only
6	6	7	copy to target only

After the rephase the target follows the new region of the search, while best still holds the 7-variable assignment, the best one so far.

Why it helps / cost. On satisfiable formulas the solver keeps steering back to its most complete partial model, which is where solutions tend to be found. The cost is copying the phase array (one byte per variable) whenever a new maximum is reached.

CaDiCaL: `backtrack.cpp` (`update_target_and_best`), `phases.cpp`, `decide.cpp`; option `target` = 1 (use target phases in stable mode only) · kissat: `backtrack.c` (`kissat_update_target_and_best_phases`, stable mode only), `phases.c`, `decide.c`; `target` = 1 · MiniSat: — · ksat port: ported (CaDiCaL and kissat)

Rephasing

Intuition. Phase saving and target phases make the search stubborn. Every so often, the solver deliberately overwrites the saved phases to try a different region: all false, all true, the best phases, a local search result, a random assignment. This is *rephasing*. Each rephase is cheap (one pass over the phase array), and the schedule mixes "explore" steps (original, inverted, flipped, random, walk) with "exploit" steps (best).

The rephase kinds, with the letter both solvers print in their log:

kind	new saved phases
O (original)	the initial phase for every variable (option `phase`, default true)
I (inverted)	the opposite of the initial phase for every variable
F (flipping)	every saved phase negated (CaDiCaL only)
# (random)	a random value per variable (CaDiCaL only)
B (best)	the best phases, where set
W (walk)	run local search starting from the current phases, keep its best assignment

CaDiCaL rephase schedule and intervals

CaDiCaL picks the schedule by mode. In stable mode with local search enabled (the default) it is

$$O, I, (B, W, O, B, W, I)^{\omega}$$

and in focused mode it is $F, (\#, B, W, F, B, W)^{\omega}$ (ω means "repeat forever"). So the first stable-mode rephases install all-true, then all-false, then the best phases, then run a walk, and so on.

The interval grows arithmetically: after the k-th rephase the next one is due $1000 \cdot (k+1)$ conflicts later (`rephaseint` = 1000). If every rephase happens exactly on time, they occur after about 1000, 3000, 6000 and 10000 conflicts.

kissat rephases only in stable mode, with the fixed cycle B, W, I, B, W, O, and spaces them by $1000 \cdot n \cdot (\log_{10}(n+9))^3$ conflicts after the n-th rephase (about 1000, 2258, 3770, 5529 for $n = 1, 2, 3, 4$). After

each rephase kissat copies the new saved phases into the target phases. In focused mode kissat has a lighter variant built into `kissat_decide_phase`: in 2 out of every 8 focused-mode episodes, every decision uses the original phase (episodes 2, 10, 18, …) or the inverted phase (episodes 4, 12, 20, …).

Why it helps / cost. Rephasing is mainly a win on satisfiable instances: it shakes the solver out of a region it keeps returning to, and the B steps bring it back to the best region if the exploration did not pay off. Except for walk, each step costs one pass over the variables.

CaDiCaL: `rephase.cpp` (`rephase`, `rephase_best`, …), options `rephase`, `rephaseint` = 1000 · kissat: `rephase.c` (`rephase_schedule`), `decide.c` (focused-mode phases), options `rephase`, `rephaseinit` = `rephaseint` = 1000 · MiniSat: — · ksat port: not ported (neither the rephase schedules nor kissat's focused-mode forced phases)

Stable and focused mode

Intuition. Two kinds of search work well on different formulas. *Focused* search restarts very often and follows the most recent conflicts closely (VMTF, Glucose-style restarts); it is good at proving unsatisfiability. *Stable* search restarts rarely and moves slowly (EVSIDS, target phases, reluctant doubling restarts); it is good at finding models. Since the solver cannot know in advance which kind of formula it has, CaDiCaL and kissat alternate between the two modes, with growing time slices.

What switches together with the mode:

	focused	**stable**
variable choice	VMTF queue	EVSIDS heap
phase	saved phase	target phase, then saved
restarts	Glucose EMA, very frequent	reluctant doubling (Luby), rare
glue averages	own set	own set (swapped on switch)

CaDiCaL calls focused mode "unstable" (the flag `stable` is false). Each mode has its own glue averages, so a switch does not mix the statistics of the two kinds of search.

When CaDiCaL switches

CaDiCaL starts in focused mode. The first switch happens after `stabilizeint` = 1000 conflicts. Each later phase is twice as long as the previous one (`stabilizefactor` = 200%):

conflicts	**mode**	**length**
0 to 1000	focused	1000
1000 to 3000	stable	2000
3000 to 7000	focused	4000
7000 to 15000	stable	8000

kissat also starts in focused mode for `modeinit` = 1000 conflicts, but measures the stable phase differently: a stable phase lasts as many *ticks* (a measure of propagation work) as the preceding focused phase used. The next focused phase gets $1000 \cdot n \cdot (\log_{10}(n+9))^4$ conflicts, where n counts the focused/stable pairs so far. Measuring stable mode in ticks keeps the propagation work spent in the two modes roughly balanced.

Why it helps / cost. One configuration does well on both satisfiable and unsatisfiable formulas, without the user having to choose. The cost is keeping both the VMTF queue and the heap around (bumping only updates the one in use; kissat refills its heap with the unassigned variables when it enters stable mode).

CaDiCaL: `restart.cpp` (`stabilizing`), `internal.cpp` (initial limits), options `stabilize`, `stabilizeint` = 1000, `stabilizefactor` = 200 · kissat: `mode.c` (`kissat_switching_search_mode`, `update_mode_limit`), options `stable` = 1, `modeinit` = `modeint` = 1000 · MiniSat: — (one mode) · ksat port: ported (CaDiCaL and kissat)

Restarts

Intuition. A restart backtracks to level 0 (or close to it, see trail reuse) while keeping all learned clauses, scores and saved phases. With phase saving, the solver rebuilds most of the old trail quickly, but in a new order determined by the current scores, so early decisions that no longer look important get replaced. The question is *when* to restart. Three schemes are in use.

Luby restarts and reluctant doubling

The Luby sequence is $1, 1, 2, 1, 1, 2, 4, 1, 1, 2, 1, 1, 2, 4, 8, \ldots$: each block repeats everything before it and then doubles the last value. It comes from a classic result (Luby, Sinclair and Zuckerman) on restarting randomized algorithms whose run-time distribution is unknown. The solver multiplies it by a base interval and restarts after that many conflicts.

> **Luby intervals in MiniSat and CaDiCaL**
>
> MiniSat (`rfirst` = 100, `rinc` = 2, Luby on by default) restarts after
>
> $$100, 100, 200, 100, 100, 200, 400, 100, \ldots$$
>
> conflicts. The cumulative restart points are 100, 200, 400, 500, 600, 800, 1200.
>
> CaDiCaL and kissat compute the same sequence incrementally with Knuth's "reluctant doubling" bit trick: keep a pair (u, v), starting with $(1, 1)$; at each step, if `(u & -u) == v` (the lowest set bit of u equals v) then $(u, v) \leftarrow (u + 1, 1)$, else $v \leftarrow 2v$. The successive values of v are exactly the Luby sequence. With base period 1024 (`reluctant` in CaDiCaL, `reluctantint` in kissat), stable mode restarts after 1024, 1024, 2048, 1024, 1024, 2048, 4096, … conflicts. When v reaches $2^{20} = 1048576$ (`reluctantmax` / `reluctantlim`), the sequence starts over.

MiniSat uses Luby restarts all the time. CaDiCaL and kissat use reluctant doubling only in stable mode, which gives long quiet stretches between restarts.

Glucose-style EMA restarts

The Glucose solver observed: when recently learned clauses have much higher glue than usual, the current trail is probably in a bad place, so restart. CaDiCaL and kissat implement this with two exponential moving averages (EMAs) of the glue of learned clauses:

- a *fast* average with window 33 (smoothing factor $\alpha = \frac{1}{33}$),
- a *slow* average with window 10^5 ($\alpha = 10^{-5}$).

Each new glue g updates an average by $x \leftarrow x + \alpha(g - x)$. A restart is triggered when $\text{fast} \geq 1.1 \cdot \text{slow}$ (`restartmargin` = 10%), and at least a minimum number of conflicts have passed since the last restart (`restartint` = 2 in CaDiCaL; kissat uses 1, plus a small logarithmic term that grows with the number of restarts).

Starting an EMA at 0 would bias it towards 0 for a long time (the slow one for about 10^5 conflicts). Both solvers use the bias correction from the ADAM optimizer: they also track β^t with $\beta = 1 - \alpha$ and report $\frac{x}{1-\beta^t}$. With this correction, the first value is exactly the first glue.

> **A burst of high-glue clauses triggers a restart**
>
> Toy version first, with $\alpha = \frac{1}{2}$ and no bias correction: glues 4, 4, 8 move the average $0 \to 2 \to 3 \to 5.5$.

Now the real parameters. The solver learns 1000 clauses of glue 4, then a run of clauses of glue 8. Both bias-corrected averages are exactly 4.000 after the first 1000 conflicts. Then:

conflict	glue	fast	slow	$1.1 \cdot \text{slow}$
1001	8	4.121	4.004	4.404
1002	8	4.239	4.008	4.409
1003	8	4.353	4.012	4.413
1004	8	4.463	4.016	4.418

At conflict 1004 the fast average passes the limit and the solver restarts. The slow average barely noticed the burst; that is its job.

Why it helps / cost. Frequent restarts let the solver throw away bad early decisions while the learned clauses keep the progress; the EMA rule restarts exactly when the clause quality drops. In stable mode the opposite is wanted, long runs that can reach a model, hence the reluctant doubling there. The cost of a restart itself is rebuilding the trail, which trail reuse reduces.

CaDiCaL: `restart.cpp` (`restarting`, `restart`), `reluctant.hpp`, `ema.hpp`, `ema.cpp`; options `restartint` = 2, `restartmargin` = 10, `emagluefast` = 33, `emaglueslow` = 1e5, `reluctant` = 1024, `reluctantmax` = 1048576 · kissat: `restart.c`, `reluctant.c`, `averages.c`, `smooth.c`; options `restartint` = 1, `restartmargin` = 10, `emafast` = 33, `emaslow` = 1e5, `reluctantint` = 1024, `reluctantlim` = 1048576 · MiniSat: `Solver.cc` (`luby`, `solve_`; `rfirst` = 100, `rinc` = 2) · ksat port: ported (Luby in MiniSat; Glucose EMA and reluctant doubling in CaDiCaL and kissat)

Trail reuse on restart

Intuition. Right after a restart, the solver makes decisions in score order. If the first few decisions on the old trail are exactly the variables it would pick again anyway, undoing and redoing them is wasted work. Trail reuse (CaDiCaL's source credits Marijn Heule) asks: "what is the next decision variable x right now?" Every decision on the old trail with a higher score (or newer VMTF stamp) than x would be picked again before x, so those levels are kept and the solver backtracks only to the first level whose decision ranks below x.

Reusing two levels

Stable mode. The trail has four decision levels with decisions a (score 9), b (score 7), c (score 4), d (score 2). After the restart the best unassigned variable is e with score 5.

Levels 1 and 2 (a, b) have scores above 5, so a full restart would pick them again before e anyway. Level 3 (c, score 4) ranks below e. The solver backtracks to level 2 instead of level 0 and keeps the assignments of levels 1 and 2.

Why it helps / cost. It saves propagation work on every restart, which matters most in focused mode where restarts happen every few conflicts. The search itself is almost the same as with a full restart. The cost is one scan over the decision levels.

CaDiCaL: `restart.cpp` (`reuse_trail`), option `restartreusetrail` = 1 · kissat: `restart.c` (`reuse_stable_trail`, `reuse_focused_trail`), option `restartreusetrail` = 1 · MiniSat: — · ksat port: ported in kissat; the CaDiCaL port always restarts to level 0

Reorder

Intuition. VSIDS and VMTF only learn from conflicts. Every so often kissat adds a static opinion from the formula itself: a literal that occurs in many short clauses is important, because assigning it false

quickly forces other literals. `reorder` computes a weight per variable from the irredundant (original, not learned) clauses and uses it to reshuffle the decision order.

The weight of a clause with s unassigned literals is $2^{-(s-2)}$: 1 for a binary clause, 1/2 for a ternary clause, 1/4 for size 4, and so on (sizes are capped at `reordermaxsize` = 100). Each literal sums the weights of the clauses it occurs in; already satisfied clauses are skipped. A variable's weight is $\max(p, n) + 2 \cdot \min(p, n)$ where p and n are the weights of its positive and negative literal, so variables that occur in both polarities count more.

Reorder weights

Clauses $C_1 = (a \vee b)$, $C_2 = (\bar{a} \vee c)$, $C_3 = \left(a \vee \bar{b} \vee c\right)$, $C_4 = \left(\bar{a} \vee \bar{b} \vee \bar{c}\right)$.

variable	p (positive)	n (negative)	weight
a	$1 + 0.5 = 1.5$ (C_1, C_3)	$1 + 0.5 = 1.5$ (C_2, C_4)	$1.5 + 2 \cdot 1.5 = 4.5$
b	1 (C_1)	$0.5 + 0.5 = 1$ (C_3, C_4)	$1 + 2 \cdot 1 = 3$
c	$1 + 0.5 = 1.5$ (C_2, C_3)	0.5 (C_4)	$1.5 + 2 \cdot 0.5 = 2.5$

New preferred order: a, b, c. In focused mode kissat sorts the variables by increasing weight (ties broken by the old stamp) and moves each to the front of the VMTF queue, so a ends up in front. In stable mode it first rescales all scores to at most 1 and then adds the weight to each score.

Why it helps / cost. It corrects the decision order when the conflict-driven scores have drifted away from the structure of the formula. The cost is one pass over all irredundant clauses plus a sort, done at level 0 every `reorderint` $\times\, n$ conflicts (linear growth, base 10000).

CaDiCaL: — · kissat: `reorder.c` (`compute_weights`, `reorder_focused`, `reorder_stable`), options `reorder` = 2 (both modes), `reorderinit` = `reorderint` = 10000, `reordermaxsize` = 100 · MiniSat: — · ksat port: not ported

Warmup

Intuition. Local search (walk, below) starts from a full assignment, usually the current decision phases. But those phases can be inconsistent with simple implications: $a = 1$ and $c = 0$ while a binary clause $(\bar{a} \vee c)$ says that a implies c. kissat's *warmup* fixes the obvious problems before walking: it runs the normal decision procedure (same variable order, same phases) and unit propagation over the whole formula, but it ignores conflicts and keeps going until every variable is assigned. Each assignment updates the saved phase. Then it backtracks to level 0 without touching target or best phases.

Warmup repairs an implied value

Clauses $C_1 = (\bar{a} \vee c)$ and $C_2 = (b \vee \bar{c} \vee d)$. Assume the decision order is a, b, c, d and no target phases are set, so decisions use the saved phases $a = 1, b = 0, c = 0, d = 1$. These falsify C_1.

Warmup decides $a@1$ (saved value 1), and C_1 forces $c@1 \leftarrow C_1$. Next it decides $\bar{b}@2$; now C_2 has b and $\bar{c}$ false, so $d@2 \leftarrow C_2$. All variables are assigned: $a = 1, b = 0, c = 1, d = 1$, which satisfies both clauses. The walker starts from this assignment instead of one with a falsified clause.

Why it helps / cost. The walker starts from fewer falsified clauses, so its flips go into real conflicts instead of repairing implications. The cost is one full round of propagation, much less than the walk itself.

CaDiCaL: — · kissat: `warmup.c` (`kissat_warmup`), `propbeyond.c` (propagation that continues after conflicts), called from `walk.c`; option `warmup` = 1 · MiniSat: — · ksat port: not ported

Random decisions

Intuition. Every heuristic so far is deterministic and self-reinforcing. A bit of randomness can break a loop where the solver keeps making the same early decisions.

MiniSat has a simple version: with probability `rnd-freq` it picks a random variable from the heap instead of the top one. The default is 0, so MiniSat makes no random decisions unless asked.

kissat uses random decision *sequences*. In focused mode (by default not in stable mode), once enough conflicts have passed, the next decision at level 1 (right after a restart or mode switch) starts a sequence: for the next few conflicts every decision picks a random unassigned variable. Phases still come from `kissat_decide_phase`. Switching into focused mode also starts (or continues) a sequence.

> **kissat random decision sequences**
> The first sequence may start after `randecinit` = 500 conflicts. The n-th sequence lasts $\lfloor 10 \cdot \log_{10}(n+9) \rfloor$ conflicts (`randeclength` = 10), which is 10 conflicts for $n = 1, 2, 3$. After the n-th sequence starts, the next may start $500 \cdot \log_{10}(n+9)$ conflicts later: 500, then 520, then 539.
>
> So in a typical focused run, about every 500 conflicts the solver spends 10 conflicts with random variable choices, then returns to VMTF.

Why it helps / cost. It diversifies the early decisions in focused mode. The cost is negligible, but it makes runs depend on the random seed (`seed`), which matters if one wants to compare traces.

CaDiCaL: — (2.1.3 has no random decisions; randomness enters only via rephasing, walk and the optional `shuffle`) · kissat: `decide.c` (`next_random_decision`, `kissat_start_random_sequence`), `propsearch.c`, `mode.c`; options `randec` = 1, `randecfocused` = 1, `randecstable` = 0 · MiniSat: `Solver.cc` (`pickBranchLit`, `rnd-freq` = 0) · ksat port: not ported (the MiniSat port has no `rnd-freq` code; upstream default 0 behaves the same)

Lucky phases

Intuition. Some formulas, even in competitions, are satisfied by trivial assignments: all variables false, all true, or "set each variable to false and propagate". Before the real search, CaDiCaL and kissat try a handful of such *lucky* assignments. Each attempt is one pass of decisions plus propagation; if it reaches a full assignment without conflict, the formula is solved.

The CaDiCaL order is:

1. all false, if every clause contains a negative literal;
2. all true, if every clause contains a positive literal;
3. forward true: for $x_1, x_2, \ldots$ in index order, decide $x_i = 1$ if still unassigned, propagate, give up on a conflict;
4. forward false, backward false, backward true (same, other value or reverse order);
5. positive Horn: for each clause not yet satisfied, set its first positive literal true and propagate, then set the remaining variables false; negative Horn symmetrically. (The name comes from Horn clauses: clauses with at most one positive literal.)

kissat does the same constant checks, then forward false, forward true, backward false, backward true (no Horn variants). Its forward/backward passes are a bit more forgiving: if a decision fails, they try the opposite value, and a failure at level 1 becomes a learned unit.

> **Lucky forward assignment**
> $C_1 = (a \vee b)$, $C_2 = \left(\overline{a} \vee \overline{b}\right)$, $C_3 = (b \vee \overline{c})$.

All false fails (C_1 has no negative literal), all true fails (C_2 has no positive literal). CaDiCaL's forward true: decide $a@1$; C_2 forces $\overline{b}@1 \leftarrow C_2$; then C_3 forces $\overline{c}@1 \leftarrow C_3$. Every variable is assigned, no conflict: the model is $a = 1, b = 0, c = 0$.

kissat's first non-constant attempt is forward false: $\overline{a}@1$, then $b@1 \leftarrow C_1$, then decide $\overline{c}@2$. Also a model: $a = 0, b = 1, c = 0$.

Why it helps / cost. On the formulas where it works, it solves them instantly. On all others it costs a few propagation passes over the formula, which is small compared to the search.

CaDiCaL: `lucky.cpp` (`lucky_phases`, `forward_true_satisfiable`, `positive_horn_satisfiable`, …), option `lucky` = 1 · kissat: `lucky.c` (`kissat_lucky`), options `lucky`, `luckyearly`, `luckylate` = 1 · MiniSat: — · ksat port: not ported

Local search (walk) and how it feeds phases

Intuition. CDCL extends a partial assignment and never violates a clause. Local search does the opposite: it starts with a full assignment that violates some clauses and repairs it by flipping single variables. It cannot prove UNSAT, but it is very fast at finding models of some satisfiable formulas. Modern CDCL solvers use it as a phase generator: run a short local search, and copy the best assignment it found (fewest violated clauses) into the saved phases.

Both CaDiCaL and kissat implement ProbSAT, using the constants from Adrian Balint's thesis:

1. Start from the current decision phases (target, else saved).
2. Pick a falsified ("broken") clause. CaDiCaL picks one at random; kissat cycles through the list of broken clauses.
3. For each literal in it, compute the *break value*: how many currently satisfied clauses would become falsified if this literal were made true (clauses where the variable's current literal is the only true one).
4. Pick a literal at random, with probability proportional to $\mathrm{cb}^{-\mathrm{break}}$, and flip it.
5. Whenever the number of broken clauses reaches a new minimum, remember the assignment. Stop at zero (a model) or when the effort limit is used up.

The constant cb is either 2 or a value fitted to the average clause length (2.5 at length 3, 2.85 at length 4, up to 7.4 at length 7, interpolated linearly); both solvers alternate between the two on successive walks.

One ProbSAT step

$C_1 = (a \vee b)$, $C_2 = (\overline{a} \vee c)$, $C_3 = \left(\overline{b} \vee \overline{c}\right)$, $C_4 = \left(\overline{a} \vee \overline{b} \vee c\right)$. Start: $a = b = c = 1$. Only C_3 is broken.

Candidates in C_3: flip b to 0 or flip c to 0.

- Flip b: C_1 is still satisfied by a, C_4 by c. Break value 0.
- Flip c: C_2 and C_4 lose their only true literal. Break value 2.

With cb = 2: weights $2^0 = 1$ and $2^{-2} = 0.25$, so b is flipped with probability $\frac{1}{1.25} = 0.8$ and c with 0.2. (With the fitted cb for average length 2.25, which is 2.375, it is 0.849 versus 0.151.) Flipping b gives $a = 1, b = 0, c = 1$, which satisfies all four clauses: the walk stops with a model. Flipping c would have broken C_2 and C_4 and the walk would continue.

How the result is used: CaDiCaL writes every new minimum straight into the saved phases (`walk_save_minimum`). kissat keeps the best assignment and, if it is better than the starting one, copies it into the saved phases at the end (`export_best_values`), then rephase copies saved into target. Either way, the next CDCL decisions follow the local search result, and if it was a model, propagation will usually confirm it quickly. Walk is triggered as one of the rephase steps (W).

Why it helps / cost. On satisfiable formulas, a few thousand flips can produce phases much better than anything CDCL found. The effort is bounded relative to the search: CaDiCaL allows `walkreleff` = 20 per mille of the search propagations (between `walkmineff` = 10^5 and `walkmaxeff` = 10^7), kissat `walkeffort` = 50 per mille of the search ticks. Learned clauses are ignored by default; the walk only looks at irredundant (original, not learned) clauses.

CaDiCaL: `walk.cpp` (`walk`, `walk_round`, `walk_pick_lit`, `walk_break_value`, `walk_save_minimum`), options `walk` = 1, `walkreleff` = 20 · kissat: `walk.c` (`kissat_walk`, `pick_literal`, `export_best_values`), option `walkeffort` = 50 · MiniSat: — · ksat port: not ported

Lookahead and cubing

Intuition. Lookahead solvers choose each decision by trying it out: for a candidate literal l, assign it, propagate, measure how much got simplified, undo. The literal whose propagation does the most work is likely a good split point. This is far too slow to run at every decision of a CDCL solver, but it is the classic way to split a hard formula into many independent sub-problems (*cube and conquer*): pick a split literal l, then split both l and $\bar{l}$ again, down to some depth. Each leaf is a *cube* (a conjunction of literals), and each cube can be solved separately, for example in parallel, by passing it as assumptions (temporary decisions forced by the caller, chapter 6) to a CDCL solver.

CaDiCaL's `lookahead` measures a probe by the trail size after propagating it. As probes it takes literals l whose negation $\bar{l}$ occurs in a binary clause, because only those trigger binary implications. A probe that propagates to a conflict is a *failed literal*: $\bar{l}$ is learned as a unit. Ties are broken by the VMTF stamp.

Choosing a split literal

$C_1 = (\bar{a} \vee b)$, $C_2 = (\bar{a} \vee c)$, $C_3 = \left(\bar{b} \vee d\right)$, $C_4 = (\bar{c} \vee \bar{e})$, $C_5 = (d \vee e \vee f)$.

Probes (negation occurs in a binary clause) and their trail sizes after propagation:

probe	propagates	trail size
a	b, c, then d (C_3), $\bar{e}$ (C_4)	5
$\bar{d}$	$\bar{b}$ (C_3), then $\bar{a}$ (C_1)	3
e	$\bar{c}$ (C_4), then $\bar{a}$ (C_2)	3
b, $\bar{b}$, c, $\bar{c}$	one more literal each	2

a wins. Cubing to depth 1 gives the two cubes (a) and $(\bar{a})$; depth 2 would run lookahead again under each cube and split each of them once more.

Why it helps / cost. Lookahead gives well-balanced splits for parallel or distributed solving, and failed literals found on the way simplify the formula. Each round propagates every probe, so it is expensive; CaDiCaL runs it only on request (`Solver::lookahead`, `Solver::generate_cubes`), never inside its normal search.

CaDiCaL: `lookahead.cpp` (`lookahead_probing`, `lookahead_generate_probes`, `generate_cubes`) · kissat: — · MiniSat: — · ksat port: not ported

The chapter at a glance

technique	CaDiCaL	kissat	MiniSat	ksat port
EVSIDS heap	stable mode	stable mode	always	ported
VMTF queue	focused mode	focused mode	—	ported

technique	CaDiCaL	kissat	MiniSat	ksat port
phase saving	yes	yes	yes	ported
target / best phases	yes	yes (stable)	—	ported
rephasing	yes	stable only	—	not ported
stable / focused switching	yes	yes	—	ported
Luby / reluctant restarts	stable	stable	always	ported
Glucose EMA restarts	focused	focused	—	ported
trail reuse	yes	yes	—	kissat only
reorder	—	yes	—	not ported
warmup	—	yes	—	not ported
random decisions	—	focused	off by default	not ported
lucky phases	yes	yes	—	not ported
walk	yes	yes	—	not ported
lookahead / cubing	on request	—	—	not ported

Managing learned clauses

Chapter 1 showed that every conflict adds a learned clause. This chapter is about the other half of that story: which learned clauses the solver throws away again, how it decides, and how it cleans up memory afterwards. The techniques here do not change **what** the solver can prove. They decide how fast it runs and whether it runs out of memory.

Why learned clauses must be pruned

A learned clause is **redundant**: it follows from the original clauses, so deleting it never changes whether the formula is SAT or UNSAT. That is what makes pruning safe. The original clauses (CaDiCaL and kissat call them **irredundant**) are never deleted by this mechanism.

Pruning is also necessary. A CDCL solver learns one clause per conflict, and modern solvers reach thousands of conflicts per second. Every stored clause costs memory, and every clause that is watched costs time in unit propagation: when a watched literal becomes false, the solver has to visit the clause. Most learned clauses are never useful again; a few are used over and over. The job is to find the few.

> **Growth without pruning**
> One clause per conflict means 10^6 learned clauses after 10^6 conflicts, usually far more than the original formula has. Each of them sits in two watch lists. Propagating a single literal may now have to visit thousands of clauses that will never become unit or conflicting, so each propagation gets slower while the search learns little from them.

All three solvers therefore run a **reduce** step from time to time: rank the learned clauses, delete the less useful part, keep the rest. They differ in how they rank. CaDiCaL and kissat use the clause's **glue** (next section); MiniSat uses a clause **activity**.

LBD (glue): a quality score for learned clauses

Intuition: look at the decision levels of a clause's literals on the current trail. Literals assigned at the same decision level are tied together: once the decision of that level is made, propagation sets all of them. A clause whose literals sit on few distinct levels is "almost unit" in many situations, because few decisions decide all of its literals. Such a clause tends to propagate and to take part in conflicts often.

The **literal block distance** (LBD) of a clause is the number of distinct decision levels among its literals. It was introduced by the Glucose solver; CaDiCaL and kissat call the same number the **glue** (the literals of one level are "glued" together). Low glue means a good clause.

> **Computing the LBD on a trail**
> Trail (decisions and propagations, see chapter 1): $a@1$, $b@1 \leftarrow R_1$, $c@2$, $d@2 \leftarrow R_2$, $e@3$, $f@3 \leftarrow R_3$, $g@4$ (R_1, R_2, R_3 are some reason clauses, not shown).
>
> Two learned clauses of the same size:
> • $E_1 = \left(\overline{b} \vee \overline{c} \vee \overline{d} \vee \overline{f}\right)$: levels $1, 2, 2, 3$, so three distinct levels, LBD $= 3$.
> • $E_2 = (\overline{a} \vee \overline{c} \vee \overline{e} \vee \overline{g})$: levels $1, 2, 3, 4$, so LBD $= 4$.
>
> This trail falsifies both clauses completely, so it is not a state the solver would continue from; we only use the trail to read off the levels of these clauses' literals.
>
> Both have four literals, but E_1 is the better clause: on this trail only three decisions matter for it, and c and d always come together.

The LBD is computed when the clause is learned. It can be recomputed later, when the clause is used again on a different trail, and then it may be smaller (see "promotion" below). It is a cheap measurement: one pass over the literals with a stamp per level.

> **Off by one in CaDiCaL and kissat**
> The value both solvers **store** for a freshly learned clause is one less than the LBD above. CaDiCaL collects all levels seen in conflict analysis and stores `glue = levels.size () - 1` (`analyze.cpp`); kissat only collects the levels below the conflict level (`deduce.c`, `learn.c`). In both cases the level of the 1st UIP literal is not counted. For the learned clause $\left(\overline{b} \vee \overline{d} \vee \overline{f}\right)$ with UIP f at level 3 on the trail above, the LBD is 3 and the stored glue is 2. A later recomputation (`recompute_glue`, `kissat_recompute_glue`) counts all levels. The thresholds below refer to the stored value.

CaDiCaL: `analyze.cpp` (`analyze_literal`, `recompute_glue`) · kissat: `deduce.c`, `learn.c`, `promote.h` · MiniSat: — (activity instead) · ksat port: CaDiCaL and kissat ports compute and store the glue exactly like upstream (stored glue = levels − 1).

Tiers: keep, keep while used, reducible

Glue alone is not enough. A clause with glue 9 that was used in the last few conflicts is still worth keeping; a clause with glue 3 that has not been touched for a long time may not be. Modern solvers combine both signals in three **tiers**:

- **tier 1**, very low glue: kept (practically) forever.
- **tier 2**, medium glue: kept as long as it was used recently.
- **tier 3**, everything else: a candidate for deletion at every reduce.

"Used" means: the clause took part in conflict analysis (it was the conflicting clause or the reason of a literal that analysis resolved on). Each clause carries a small `used` counter for this. Using the clause sets the counter, and every reduce decrements it. So the counter answers "was this clause used in the last one or two reduce intervals?".

In CaDiCaL the tier limits are fixed options: `reducetier1glue` $= 2$ and `reducetier2glue` $= 6$ (`options.hpp`). Tier-1 membership is a flag on the clause, `keep`, set when the clause is created with glue ≤ 2 (`clause.cpp`). A new learned clause starts with `used` $= 2$ if its glue is ≤ 6 (tier 2) and `used` $= 1$ otherwise (`analyze.cpp`: `res->used = 1 + (glue <= opts.reducetier2glue)`).

kissat computes its tier limits dynamically; that is the subject of the kissat section below.

CaDiCaL: `options.hpp` (`reducetier1glue`, `reducetier2glue`), `clause.cpp` (`keep`), `clause.hpp` (tier comment) · kissat: `tiers.c`, `options.h` (`tier1`, `tier2`) · MiniSat: — · ksat port: CaDiCaL port uses the limits 2 and 6; kissat port has dynamic tiers.

The reduce step in CaDiCaL

A reduce is triggered by the conflict counter. The first one happens after `reduceint` $= 300$ conflicts; after the k-th reduce the next one comes $300 \cdot (k+1)$ conflicts later, so the intervals grow (scaled up further by $\log_{10}(n/10^4)$ when there are more than 10^5 irredundant clauses, `reduce.cpp`). Growing intervals mean the solver keeps more and more learned clauses over time.

The reduce itself (`mark_useless_redundant_clauses_as_garbage`) works like this:

1. Skip irredundant clauses, clauses already marked as garbage, and reason clauses.
2. Read `used`, then decrement it. If it was non-zero, keep the clause (recently used).
3. If the clause has the `keep` flag (tier 1), keep it.
4. All remaining clauses are **candidates**. Sort them from least to most useful: larger glue first, and for equal glue larger size first (a stable sort, so among equal clauses the older one comes first and goes first).
5. Mark the first `reducetarget` $= 75\%$ of the candidates as garbage.

Marking as garbage does not free memory yet; garbage collection (later in this chapter) does.

One CaDiCaL reduce, by hand

Nine learned clauses (glue is the stored value):

clause	glue	size	used	keep	reason?	outcome
D_1	2	3	0	yes	no	kept (tier 1)
D_2	5	7	2	no	no	kept (used), used $\to$ 1
D_3	5	6	0	no	no	candidate
D_4	9	11	1	no	no	kept (used), used $\to$ 0
D_5	9	10	0	no	yes	kept (reason)
D_6	9	12	0	no	no	candidate
D_7	7	8	0	no	no	candidate
D_8	12	15	0	no	no	candidate
D_9	7	10	0	no	no	candidate

Five candidates, sorted least useful first: D_8 (glue 12), D_6 (glue 9), D_9 (glue 7, size 10), D_7 (glue 7, size 8), D_3 (glue 5). The target is $\lfloor 0.75 \cdot 5 \rfloor = 3$, so D_8, D_6 and D_9 are deleted; D_7 and D_3 survive this round.

Next reduce, assuming none of the clauses is used in between and D_5 is still a reason: D_2 reads `used` $= 1$ and is kept one more time (then 0); D_4 now reads 0 and becomes a candidate. Candidates D_4, D_7, D_3; target $\lfloor 0.75 \cdot 3 \rfloor = 2$; D_4 and D_7 are deleted. This is how a tier-2 clause (D_2, glue 5) gets two reduce rounds of grace while a tier-3 clause (D_4) gets one.

Why it helps: the glue ranking deletes the clauses that are least likely to propagate, and the `used` counter rescues clauses that are bad on paper but useful right now. Cost: one pass over all learned clauses plus a sort, a small fraction of the run time.

CaDiCaL also has a **flush** mode (option `flush`, off by default) that deletes every unused learned clause regardless of glue.

CaDiCaL: `reduce.cpp` (`reducing`, `mark_useless_redundant_clauses_as_garbage`, `reduce`), `options.hpp` (`reduceint` 300, `reducetarget` 75) · ksat port: `CaDiCaL.kt` `reduce()`. The port follows its shadow reference `ksat-extra/shadow/cadical-c/cadical_trace.cc`, which differs from upstream 2.1.3 here: no `keep` flag, a kissat-style used test (glue ≤ 2 kept only while `used` > 0, glue ≤ 6 kept while `used` ≥ 1; a use sets `used` to 2, a new learned clause starts at 1), and the next reduce after $25 \cdot \sqrt{\text{conflicts}}$ conflicts (constant `OPT_reduceint = 25` in the reference). The glue/size ranking and the 75% target are as upstream.

Dynamic tiers and reduce in kissat

kissat asks: which glues do the clauses that actually get used have? It keeps a histogram `used[glue]`, incremented every time conflict analysis uses a learned clause of that glue (`deduce.c`, separately for stable and focused mode). `tiers.c` picks the limits from this histogram:

- tier1 = the smallest glue g such that clauses with glue $\leq g$ account for at least `tier1relative` $= 50$ % of all uses;
- tier2 = the smallest glue g such that clauses with glue $\leq g$ account for at least `tier2relative` $= 90$ % of all uses.

If the histogram is empty, the static defaults `tier1` $= 2$ and `tier2` $= 6$ are used.

The limits are recomputed at every reduce, and in addition on a separate conflict interval (`update_tier_limits` in `analyze.c`): that interval starts at 2 conflicts and doubles after each recomputation, up to $2^{16} = 65536$ conflicts. So early in the run the limits follow the histogram closely. (In kissat 4.0.4 the interval is checked in the branch of `kissat_analyze` that restarts analysis after on-the-fly strengthening, chapter 1, so upstream it only fires on such conflicts. The ksat port, following its shadow reference, checks it after every conflict.)

Either way the tiers adapt to the instance: on a problem where good clauses naturally have glue 10, tier 1 moves up instead of declaring everything “bad”.

kissat tier limits from a use histogram

Suppose learned clauses were used 100 times in total, distributed by glue:

glue	1	2	3	4	5	6	7	8	9
uses	10	25	20	15	10	8	5	4	3
accumulated	10	35	55	70	80	88	93	97	100

50% of 100 is reached first at glue 3 (55), so tier1 $= 3$. 90% is reached first at glue 7 (93), so tier2 $= 7$.

The reduce rule (`collect_reducibles` in `reduce.c`) then reads each learned clause’s `used` counter (5 bits, set to `MAX_USED` $= 31$ whenever the clause is used, decremented at every reduce) and decides:

- reason clauses are skipped;
- glue $\leq$ tier1 and `used` > 0: keep;
- glue $\leq$ tier2 and `used` ≥ 30: keep (used since the last or the one before last reduce);
- everything else is a candidate, sorted by glue then size, largest first.

The fraction deleted is not fixed. With `reducelow` $= 50\%$ and `reducehigh` $= 90\%$ it is $90 - 40/\log_{10}(\text{reductions} + 9)$ percent: 50% at the first reduce, 70% at the 91st, slowly approaching 90%. The interval between reduces is `reduceint` $\cdot \sqrt{\text{reductions}}$ conflicts with `reduceint` $= 1000$.

One kissat reduce with tier1 = 3, tier2 = 7

clause	glue	size	used	reason?	outcome
K_1	2	4	31	no	kept (tier 1, used)
K_2	3	5	0	no	candidate (tier 1 but unused)
K_3	6	9	31	no	kept (tier 2, recently used)
K_4	6	8	29	no	candidate (tier 2, not recent)
K_5	9	14	31	no	candidate (tier 3)
K_6	12	20	0	yes	kept (reason)

Candidates sorted: K_5, K_4, K_2. At the first reduce the fraction is 50%, so $\lfloor 3 \cdot 0.5 \rfloor = 1$ clause is deleted: K_5. Note that K_5 was used recently and is still the first to go: in kissat, tier 3 gets no protection from `used` at all.

Two consequences of the counter: a tier-1 clause survives 31 reduces without being used, a tier-2 clause survives 2. So kissat's tier 1 is "keep for a very long time", not "forever".

Binary learned clauses never take part: kissat stores them only in the watch lists, not in the clause arena, and `collect_reducibles` only scans the arena.

Which tier limit is used

kissat keeps separate limits for stable and focused mode, but in 4.0.x the macros in `internal.h` read `TIER1` from the focused-mode slot and `TIER2` from the stable-mode slot (the per-mode variant sits in an `#if 0` block). The ksat port reproduces this.

kissat: `tiers.c` (`compute_tier_limits`), `reduce.c` (`collect_reducibles`, `mark_less_useful_clauses_as_garbage`), `deduce.c` (`mark_clause_as_used`), `options.h` (`tier1relative` 500, `tier2relative` 900, `reducelow` 500, `reducehigh` 900, `reduceint` 1000, per mille where applicable) · ksat port: `Kissat.kt` `computeTierLimits`, `reduce()`, same rules and fractions.

Activity-based reduceDB in MiniSat

MiniSat 2.2.0 predates LBD. It ranks learned clauses by **activity**, the same idea as VSIDS for variables (chapter 2): every time a learned clause takes part in conflict analysis, its activity grows by `cla_inc`, and `cla_inc` itself grows by a factor $1/0.999$ after every conflict (`clause_decay` $= 0.999$). So recent use counts more than old use. Activities are rescaled by 10^{-20} when one exceeds 10^{20}.

`reduceDB` is called when the number of learned clauses (minus the number of assigned literals) reaches `max_learnts`. That limit starts at one third of the number of original clauses and grows by a factor 1.1 at increasingly spaced conflict counts.

The rule (`Solver.cc`):

1. Sort the learned clauses: non-binary clauses by increasing activity, binary clauses last.
2. Let n be the number of learned clauses and `extra_lim` $=$ `cla_inc` $/n$.
3. Walk the sorted list. Delete clause i if it has more than 2 literals, is not **locked** (the reason of a current assignment), and either lies in the first half ($i < n/2$) or has an activity below `extra_lim`.

One MiniSat reduceDB

Eight learned clauses, `cla_inc` $= 8$, so `extra_lim` $= 8/8 = 1.0$:

sorted pos.	clause	size	activity	outcome
0	M_7	4	0.3	kept (locked)
1	M_1	3	0.5	deleted (first half)
2	M_4	4	0.9	deleted (first half)
3	M_8	5	0.95	deleted (first half)
4	M_5	6	0.97	deleted (activity < 1.0)
5	M_3	5	4.0	kept
6	M_6	3	7.0	kept
7	M_2	2	0.1	kept (binary)

M_2 has the lowest activity of all but is binary, so it is sorted last and never deleted. M_5 is in the second half but its activity is below `extra_lim`.

Compared with the glue-based reduce: activity measures "used recently", which CaDiCaL and kissat cover with the `used` counter, but it has no notion of how close to unit a clause typically is. Glucose showed that LBD is the better predictor, which is why later solvers switched.

MiniSat: `Solver.cc` (`reduceDB`, `reduceDB_lt`, `search` for the trigger), `Solver.h` (`claBumpActivity`, `locked`) · ksat port: `MiniSat.kt` `reduceDB()`, same rule, with a stable insertion sort using the same comparator; clause activity precision is a knob (32-bit float like upstream by default).

The used flag and promotion

When conflict analysis resolves on a learned clause, CaDiCaL and kissat do two things with it: mark it as used, and recompute its glue on the **current** trail. The trail has changed since the clause was learned, and the clause's literals may now sit on fewer levels. If the new glue is smaller, the clause gets the new glue; this is called **promotion**, because it can move the clause into a better tier.

In CaDiCaL (`bump_clause` in `analyze.cpp`, `promote_clause` in `clause.cpp`):

- `used` is set to 1;
- clauses already marked `keep` are left alone;
- if the recomputed glue is smaller: new glue ≤ 2 sets `keep` (promoted to tier 1); a move from above 6 to ≤ 6 sets `used` $= 2$ (promoted to tier 2);
- otherwise, a tier-2 clause that was already used gets `used` $= 2$.

In kissat (`mark_clause_as_used` in `deduce.c`, `kissat_promote_clause` in `promote.c`) `used` is set to 31, and promotion only lowers the stored glue. Since tiers are derived from the glue at each reduce, that is enough. The use is also counted in the histogram that drives the dynamic tiers.

A clause promoted from tier 2 to tier 1 (CaDiCaL)

$D = \left(\overline{a} \vee \overline{b} \vee \overline{c} \vee \overline{d}\right)$ was learned on a trail with $a@1$, $b@2$, $c@3$ and the UIP d at level 4. Four distinct levels, stored glue $4 - 1 = 3$: tier 2, no `keep`, `used` $= 2$.

Much later the trail is $a@1$, $b@1 \leftarrow R_4$, $c@2$, $\overline{d}@2 \leftarrow D$ (R_4 some reason clause), and a conflict analysis resolves on d, so it uses D. Recomputed glue: levels $1, 1, 2, 2$, so 2. Since $2 < 3$ and $2 \leq 2$, D gets glue 2 and the `keep` flag. From now on no reduce will delete it.

Why it helps: the glue measured at learning time is a single sample. A clause that keeps showing up in conflicts with a low glue is evidence that it is a good clause, and promotion makes the ranking reflect that. Cost: one pass over the clause per use, with a stamp per level.

CaDiCaL: `analyze.cpp` (`bump_clause`, `recompute_glue`), `clause.cpp` (`promote_clause`) · kissat: `deduce.c`, `promote.c`, `promote.h` · MiniSat: `claBumpActivity` plays the "used" role, no promotion · ksat port: CaDiCaL and kissat ports recompute the glue and promote; in the CaDiCaL port every use sets `used` to 2, and promotion only lowers the glue (no `keep` flag, see the reduce section).

Protecting reason clauses

A learned clause can be the reason of a literal that is currently on the trail. Conflict analysis may need that reason later: it walks backwards through the trail and resolves with the reason of each literal it meets. If the clause had been deleted, the reason pointer would point to freed memory.

So before a reduce marks anything as garbage, CaDiCaL walks the trail and sets a `reason` flag on every clause that is the reason of an assigned literal (`protect_reasons` in `collect.cpp`); kissat does the same with `kissat_mark_reason_clauses` (`trail.c`). The reduce skips flagged clauses, and the flags are cleared again after garbage collection. MiniSat does not keep a flag; it checks `locked(c)`: clause c is locked if its first literal is true and its variable's reason is c (the propagated literal always sits at position 0).

> **A reason clause that must survive**
> Trail $a@1$, $c@2$, $\overline{d}@2 \leftarrow D'$ with the learned clause $D' = \left(\overline{d} \vee \overline{a} \vee \overline{c}\right)$. Even if D' has the worst glue and has never been used, it is not deleted while $\overline{d}$ stays on the trail: a conflict at level 2 that involves d would resolve with D'. After a backtrack below level 2 it is an ordinary candidate again. In the CaDiCaL example above, D_5 is such a clause.

Root-level literals are the exception. Conflict analysis skips literals assigned at level 0 (they are facts), so their reasons are not needed. That is why MiniSat's `removeClause` may delete a locked clause at level 0: it simply clears the variable's reason first.

CaDiCaL: `collect.cpp` (`protect_reasons`, `unprotect_reasons`) · kissat: `trail.c` (`kissat_mark_reason_clauses`, `kissat_unmark_reason_clauses`) · MiniSat: `Solver.h` (`locked`), `Solver.cc` (`removeClause`) · ksat port: all three ports protect reasons.

Root-level simplification

Assignments at decision level 0 are permanent: they follow from the formula without any decision (units, see chapter 1). Every clause can be simplified against them:

- a clause containing a literal that is true at level 0 is **satisfied forever** and can be deleted;
- a literal that is false at level 0 can be removed from its clause.

Both are safe: the formula stays equivalent under the permanent assignments.

> **Simplifying with the root units a = 1, b = 0**
> - $(a \vee c \vee d)$: a is true, the clause is deleted.
> - $(b \vee c \vee \overline{e})$: b is false, the clause becomes $(c \vee \overline{e})$.
> - $(\overline{a} \vee d \vee e \vee f)$: $\overline{a}$ is false, it becomes $(d \vee e \vee f)$.
> - $\left(\overline{b} \vee g\right)$: $\overline{b}$ is true, the clause is deleted.
> - $(c \vee d)$: unchanged.
>
> Two clauses fewer, two clauses shorter. Each deleted clause also leaves two watch lists, so propagation gets cheaper.

The three solvers do this at different moments:

- CaDiCaL: `mark_satisfied_clauses_as_garbage` (`collect.cpp`), run at the start of every reduce and garbage collection, but only if new root units appeared since the last time. It marks satisfied clauses as garbage and removes falsified literals (`remove_falsified_literals`). This covers original and learned clauses.
- kissat: the sparse garbage collection (next section) does it while it copies clauses: a clause with a root-true literal is dropped, root-false literals are not copied.
- MiniSat 2.2.0: `simplify()` runs at level 0 when there are new root assignments and enough propagations have happened since the last call (`simpDB_props`). It only deletes satisfied clauses (learned ones always, original ones if `remove_satisfied` is on); falsified literals stay in their clauses.

CaDiCaL: `collect.cpp` (`mark_satisfied_clauses_as_garbage`, `remove_falsified_literals`) · kissat: `collect.c` (`sparse_sweep_garbage_clauses`) · MiniSat: `Solver.cc` (`simplify`, `removeSatisfied`) · ksat port: MiniSat port has `simplify`/`removeSatisfied` (it runs on every new root assignment, the `simpDB_props` throttle is not ported, and following its shadow reference it also trims root-false literals beyond the two watched positions). The CaDiCaL and kissat ports drop root-satisfied clauses and root-false literals only when a clause is added, not during search.

Garbage collection and arena compaction

Marking a clause as garbage only flags it. **Garbage collection** actually removes it: take the clause out of the watch lists, free its memory, and fix everything that refers to clauses.

The solvers keep clauses in an **arena**: one big array, where each clause is a header followed by its literals, and a clause reference is just an offset into that array. This is compact and cache friendly, and offsets are smaller than pointers. The price: deleting clauses leaves holes. Collection **compacts** the arena by sliding the surviving clauses together, which changes their offsets, so every watch and every reason reference must be updated.

Compacting an arena (illustrative layout: 1 header word + 1 word per literal)

clause	literals	garbage?	offset before	offset after
A_1	3	no	0	0
A_2	4	yes	4	–
A_3	2	no	9	4
A_4	5	yes	12	–
A_5	3	no	18	7

The arena shrinks from 22 to 11 words. A watch entry that said “clause at offset 18” must now say 7; the same for a reason pointer of a literal propagated by A_5.

How each solver does it:

- MiniSat: deleting a clause only adds its size to a “wasted” counter. When wasted memory exceeds `garbage_frac` $= 20\%$ of the arena (`checkGarbage`), `garbageCollect` allocates a fresh arena and `relocAll` copies every live clause over, rewriting watches, reasons, the learned list and the original list.
- CaDiCaL: allocates new clauses individually and, from the second collection on (`arenaing`), copies all surviving clauses into a fresh arena (`copy_non_garbage_clauses`). The copy order is chosen for locality (`arenatype` $= 3$: clauses watched by the same literal end up next to each other, in variable queue order), which makes propagation faster. Otherwise garbage is deleted in place (`delete_garbage_clauses`).
- kissat: one arena for all large clauses. The sparse collection slides clauses down in place, dropping garbage and root-satisfied clauses and root-false literals in the same pass, then rebuilds the

watches. Learned clauses are kept at the end of the arena, so a reduce only sweeps from the first reducible clause onwards.

Why it helps: memory is returned, and the surviving clauses are packed together so propagation touches fewer cache lines. Cost: a pass over all clauses and all watch lists, which is why it is only done after a reduce or when enough memory is wasted.

CaDiCaL: `collect.cpp` (`garbage_collection`, `copy_non_garbage_clauses`, `delete_garbage_clauses`), `arena.hpp` · kissat: `collect.c` (`kissat_sparse_collect`, `sparse_sweep_garbage_clauses`) · MiniSat: `Solver.cc` (`garbageCollect`, `relocAll`), `Solver.h` (`checkGarbage`) · ksat port: not ported as such; each port uses its own memory layout (parallel arrays indexed by clause handle). Deleted clauses are unwatched and dropped from the clause lists (CaDiCaL, MiniSat) or just left marked (kissat), and handles never move.

Variable compaction

Over time many variables become **inactive**: fixed at the root, eliminated by preprocessing (chapter 4), or replaced by an equivalent variable (chapter 5). They no longer appear in any clause, but every per-variable array (values, levels, reasons, scores, watch lists, the decision heap or queue) still has a slot for them. **Compaction** renumbers the active variables into a contiguous range $1 \ldots n'$ and shrinks all those arrays.

Fixed variables get a special treatment in both CaDiCaL and kissat: the first fixed variable keeps one slot, and every other fixed variable is mapped onto it, positively if it has the same value and negated otherwise. The external variable numbers the user sees do not change; only the internal map from external to internal variables is rewritten.

> **Compacting 8 variables**
> Variables $1 \ldots 8$; variable 3 is fixed true, 6 is fixed false, 5 is eliminated.
>
> | | | | | | | | | |
> |---|---|---|---|---|---|---|---|---|
> | **old** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
> | **new** | 1 | 2 | 3 | 4 | – | $\overline{3}$ | 5 | 6 |
>
> Six internal variables instead of eight. Variable 6 maps to the negation of the new variable 3: new 3 is true, so $\overline{3}$ is false, which is exactly 6′s value.

When it runs: CaDiCaL checks `compacting()` at level 0 and compacts when at least `compactmin` $= 100$ variables and at least `compactlim` $= 100$ per mille (10%) of all variables are inactive, at most every `compactint` $= 2000$ conflicts. kissat decides at each reduce: if more than `compactlim` $= 10\%$ of the variables are inactive, the sparse collection also maps all literals while it copies the clauses.

Why it helps: smaller arrays, better cache use, and the heap or queue no longer carries dead entries. It matters most after heavy preprocessing, which can remove a large share of the variables.

CaDiCaL: `compact.cpp` (`compacting`, `compact`, `Mapper`), `options.hpp` (`compact`, `compactint`, `compactlim`, `compactmin`) · kissat: `compact.c` (`kissat_compact_literals`), `collect.c` (`kissat_compacting`), `options.h` (`compactlim`) · MiniSat: — · ksat port: not ported (the ports run without inprocessing, so few variables become inactive).

Simplifying the formula I: removing clauses and variables

Chapters 1–3 took the formula as given and made the search fast. This chapter changes the formula itself: it deletes clauses that are not needed, shortens clauses, and removes whole variables. A smaller formula means fewer watches to visit, fewer clauses to propagate and fewer variables to decide on.

All techniques here are built from one operation, resolution, plus some bookkeeping so the solver can still report a correct model at the end.

Preprocessing and inprocessing

Preprocessing means simplifying the formula once, before the first decision. *Inprocessing* means stopping the CDCL search from time to time (at decision level 0), simplifying the current formula including what was learned so far, and then continuing the search.

- MiniSat 2.2.0 only preprocesses. `SimpSolver::solve_` calls `eliminate()` before the CDCL search starts (the core solver from chapter 1 never simplifies on its own).
- CaDiCaL mostly inprocesses. Its main loop in `internal.cpp` checks `subsuming ()`, `eliminating ()` and `conditioning ()` between conflicts, exactly like it checks `restarting ()` or `reducing ()`. Optional preprocessing rounds (`preprocess_round`) run `probe`, `elim` and `condition` before the search.
- kissat 4.0.x does both. `preprocess.c` runs initial rounds of probing (in `probe.c` this includes factorization from this chapter and several techniques from chapter 5) and the optional fast elimination of this chapter. `eliminate.c` is scheduled as inprocessing during search.

Inprocessing is more powerful than preprocessing because units and binary clauses learned during search make more simplifications possible. It is also harder to get right, because the learned clauses, the watches and the trail all have to stay consistent with a formula that keeps changing.

Two words we need throughout:

- Two formulas are **equivalent** if they have exactly the same models. Removing a subsumed clause (below) keeps the formula equivalent.
- Two formulas are **equisatisfiable** if one is satisfiable exactly when the other one is. Removing a variable or a blocked clause only keeps the formula equisatisfiable: the simplified formula may have models that are not models of the original. Those models are repaired at the end (section “Model reconstruction”), which is why every technique below that is only equisatisfiable writes something onto a reconstruction stack.

Resolution, the basic step

Take two clauses that contain the same variable with opposite signs, one with a and one with $\overline{a}$. If $a = 1$, the second clause needs one of its other literals; if $a = 0$, the first clause does. Either way, one of the “other” literals of the two clauses must be true. That new clause is the **resolvent** on a (also written $C_1 \otimes C_2$ on a), and a is called the *pivot*.

> **Resolving two clauses**
> $C_1 = (a \vee b)$ and $C_2 = (\overline{a} \vee c)$. The resolvent on a is $(b \vee c)$. Check: if $b = 0$ then C_1 forces $a = 1$, then C_2 forces $c = 1$. So $b \vee c$ holds in every model of $C_1 \wedge C_2$.
>
> Resolving $C_1 = (a \vee b)$ with $C_3 = \left(\overline{a} \vee \overline{b}\right)$ on a gives $\left(b \vee \overline{b}\right)$, which is always true. Such a resolvent is **tautological**: it carries no information and is thrown away.

You already met resolution in chapter 1: conflict analysis is a chain of resolutions along the trail. Here it is used the other way around, not to learn one clause but to rewrite the formula. A resolvent is always implied by its two parents, so adding it never changes the models.

Subsumption and strengthening

Clause C **subsumes** clause D if every literal of C also appears in D. Then D is useless: whenever C is satisfied, so is D. Deleting D keeps the formula equivalent.

A close relative is **self-subsuming resolution** (also called *strengthening*): if C and D clash on exactly one literal, and the rest of C is contained in D, then the resolvent of C and D is D minus one literal. That resolvent subsumes D, so we can simply drop the literal from D.

> **Subsumption and self-subsuming resolution**
> $C_1 = (a \vee b)$, $C_2 = (a \vee b \vee c)$, $C_3 = (\overline{a} \vee b \vee c)$, $C_4 = \left(\overline{b} \vee d\right)$.
>
> - C_1 subsumes C_2 (both a and b are in C_2). Delete C_2.
> - C_1 and C_3 clash only on a, and the rest of C_1 (just b) is in C_3. The resolvent on a is $(b \vee c)$, which is C_3 without $\overline{a}$. So C_3 is strengthened to $(b \vee c)$.
>
> The result $(a \vee b), (b \vee c), \left(\overline{b} \vee d\right)$ has exactly the same models as the original four clauses.

There are two directions to search:

- **Forward** subsumption starts from a clause D and asks "is D subsumed (or strengthened) by some other clause?". It walks the *occurrence lists* of all literals of D looking for smaller clauses. The occurrence list of a literal is the list of all clauses containing it; unlike watches (chapter 1), every literal of every clause is indexed.
- **Backward** subsumption starts from a (usually new or short) clause C and asks "which other clauses does C subsume?". It only needs the occurrence list of one literal of C, the one with the fewest occurrences, because any subsumed clause must contain all of C.

MiniSat uses backward subsumption: `backwardSubsumptionCheck` pops clauses from a `subsumption_queue` and uses a 32-bit "abstraction" signature per clause to rule out most candidates without comparing literals. CaDiCaL's main pass in `subsume.cpp` is forward (its header comment explains why: only one literal per clause has to be connected), and it also runs on learned clauses that are likely to be kept. During elimination CaDiCaL additionally checks new resolvents backward (`backward.cpp`). kissat uses forward subsumption and strengthening during elimination (`forward.c`).

Why it helps: subsumed clauses are pure overhead in propagation and in occurrence counts, and shorter clauses propagate earlier. It is also the cleanup step after variable elimination, which tends to produce resolvents that subsume each other. The cost is the occurrence-list traversal, so all three solvers limit clause length and occurrence counts (for example kissat's `subsumeclslim` and `subsumeocclim`).

CaDiCaL: `subsume.cpp` (forward, `strengthen_clause`), `backward.cpp` (eager backward during elim), on-the-fly strengthening in `elim.cpp` · kissat: `forward.c` · MiniSat: `SimpSolver.cc` (`backwardSubsumptionCheck`, `strengthenClause`) · ksat port: not ported

Bounded variable elimination

Variable elimination (also called DP resolution, after Davis and Putnam) removes a variable x completely: take every clause containing x, resolve it with every clause containing $\overline{x}$, keep all non-tautological resolvents, and delete all original clauses that mention x. The result is equisatisfiable with the original.

The catch: with p clauses containing x and n containing $\overline{x}$ you get up to $p \cdot n$ resolvents but only remove $p + n$ clauses. Doing this blindly blows up the formula. **Bounded** variable elimination (BVE, popularized by the SatELite preprocessor) only eliminates x if the formula does not get bigger:

$$\text{non-tautological resolvents} \leq p + n + \text{bound}$$

with bound $= 0$ meaning "no more clauses than before". If the check fails, nothing is changed.

BVE that pays off

$C_1 = (x \vee a)$, $C_2 = (\overline{x} \vee b)$, $C_3 = (\overline{x} \vee c)$, plus other clauses that do not mention x.

Here $p = 1$ and $n = 2$. The resolvents are C_1 with C_2: $(a \vee b)$, and C_1 with C_3: $(a \vee c)$. Two resolvents $\leq 1 + 2$, so x is eliminated: the three clauses are replaced by $(a \vee b), (a \vee c)$. The formula lost a variable and a clause.

BVE that does not pay off

$(x \vee a), (x \vee b), (x \vee c), (\overline{x} \vee d), (\overline{x} \vee e), (\overline{x} \vee f)$.

Now $p = n = 3$. Every pair gives a non-tautological resolvent: $(a \vee d), (a \vee e), ..., (c \vee f)$, nine in total. Nine $>$ six, so with bound 0 the solver leaves x alone. (Eliminating it would still be sound, the formula just grows.)

In the examples all resolvents happened to be non-tautological. In real formulas many pairs are tautological and do not count, and that is what makes BVE so effective on encodings with lots of auxiliary variables (see the section on gates).

The bound is not fixed. CaDiCaL (`elim.cpp`, `elim_resolvents_are_bounded`) and kissat (`resolve.c`, `eliminate.c`) start with 0 and, after an elimination round has tried all candidates, increase it 0, 1, 2, 4, 8, 16 (`elimboundmax` / `eliminatebound`, both 16). MiniSat uses `grow` (default 0) and never changes it. All three also refuse to create very long resolvents (`elimclslim` 100 in CaDiCaL, `eliminateclslim` 100 in kissat, `cl-lim` 20 in MiniSat). CaDiCaL and kissat also skip variables with too many occurrences (`elimocclim`, `eliminateocclim`).

Why it helps: on structured instances BVE often removes a large fraction of the variables. Fewer variables means fewer decisions, and the resolvents are usually shorter and denser. The costs are the occurrence lists it needs (CDCL search only needs watches), the effort of trying candidates, and the reconstruction bookkeeping: x is gone, but the user still wants a value for x at the end.

CaDiCaL: `elim.cpp` (+ `backward.cpp`) · kissat: `eliminate.c`, `resolve.c`, `weaken.c` · MiniSat: `SimpSolver.cc` (`eliminateVar`, `merge`) · ksat port: not ported

Fast elimination (fastel)

kissat 4.0.x has a second, cheaper elimination pass in `fastel.c`, called only from the initial preprocessing in `preprocess.c`. It looks only at variables with few occurrences (`fasteloccs` 100) and uses a different rule from the "not bigger" bound above: with p positive and n negative occurrences,

- if $p \cdot n \leq 8$ (`fastelim`), eliminate right away without even counting;
- otherwise count non-tautological resolvents and eliminate if there are at most 8.

While collecting the occurrences it also drops satisfied clauses and runs a quick forward subsumption check (`fastelsub`), and it repeats for up to `fastelrounds` (4) rounds.

fastel accepts what BVE would reject

$(x \vee a), (x \vee b), (\overline{x} \vee c), (\overline{x} \vee d), (\overline{x} \vee e), (\overline{x} \vee f)$.

$p = 2$, $n = 4$, $p \cdot n = 8 \leq 8$, so fastel eliminates x immediately. The result has 8 resolvents ($(a \vee c), ..., (b \vee f)$) in place of 6 clauses. BVE with bound 0 would have refused ($8 > 2 + 4$).

So "fast" means: no scheduling heap, no gate detection, only a small fixed cap on resolvents, in exchange for accepting a little growth. In the kissat 4.0.4 source the option `fastel` is off by default (`options.h`), so you only get it with `--fastel=1`.

CaDiCaL: — · kissat: `fastel.c` (called from `preprocess.c`) · MiniSat: — · ksat port: not ported

Gates and definitions

Formulas produced from circuits or other encodings are full of **definitions**: auxiliary variables whose value is a fixed function of other variables. The common ones and their clause patterns:

Gate	Clauses
Equivalence $x \Leftrightarrow a$	$(\overline{x} \vee a), (x \vee \overline{a})$
AND $x \Leftrightarrow a \wedge b$	$(\overline{x} \vee a), (\overline{x} \vee b), \left(x \vee \overline{a} \vee \overline{b}\right)$
XOR $x \Leftrightarrow a \oplus b$	$(\overline{x} \vee a \vee b), \left(\overline{x} \vee \overline{a} \vee \overline{b}\right), (x \vee \overline{a} \vee b), \left(x \vee a \vee \overline{b}\right)$
ITE $x \Leftrightarrow$ (if c then a else b)	$(\overline{x} \vee \overline{c} \vee a), (\overline{x} \vee c \vee b), (x \vee \overline{c} \vee \overline{a}), \left(x \vee c \vee \overline{b}\right)$

Why gates matter for elimination: split the clauses with x into the **gate clauses** G (the definition) and the rest R. One can show that it is enough to resolve gate clauses against non-gate clauses. Gate-with-gate resolvents are tautological for these gates, and non-gate-with-non-gate resolvents are implied by the others. Fewer resolvents means the bound is met more often.

> **An AND gate makes elimination possible**
> Gate $x \Leftrightarrow a \wedge b$: $G_1 = (\overline{x} \vee a)$, $G_2 = (\overline{x} \vee b)$, $G_3 = \left(x \vee \overline{a} \vee \overline{b}\right)$. Other clauses with x: $R_1 = (x \vee c)$, $R_2 = (x \vee f)$, $R_3 = (\overline{x} \vee d)$, $R_4 = (\overline{x} \vee e)$. Seven clauses.
>
> **Plain BVE:** positive $\{G_3, R_1, R_2\}$, negative $\{G_1, G_2, R_3, R_4\}$, 12 pairs. Only G_3 with G_1 and G_3 with G_2 are tautological, leaving 10 resolvents. $10 > 7$: rejected.
>
> **Gate-aware BVE:** only G against R:
> - G_3 with R_3, R_4: $\left(\overline{a} \vee \overline{b} \vee d\right), \left(\overline{a} \vee \overline{b} \vee e\right)$
> - R_1 with G_1, G_2: $(a \vee c), (b \vee c)$; R_2 with G_1, G_2: $(a \vee f), (b \vee f)$
>
> 6 resolvents ≤ 7: eliminated. The skipped resolvents such as $(c \vee d)$ (from R_1 and R_3) follow from the six: if $c = 0$ then $a = b = 1$, so $d = 1$.

You can read the six resolvents as "substitute the definition of x into R": R_1 said $x \vee c$, i.e. $(a \wedge b) \vee c$, which is $(a \vee c) \wedge (b \vee c)$. For an equivalence $x \Leftrightarrow a$ this is literally replacing x by a: $(x \vee c)$ becomes $(a \vee c)$.

How the solvers find gates:

- CaDiCaL (`gates.cpp`, `find_gate_clauses`) tries `find_equivalence`, `find_and_gate`, `find_if_then_else` and `find_xor_gate` by pattern matching on binary and ternary clauses. During resolution it skips pairs where both clauses are gate clauses or both are not (`c->gate == d->gate` in `elim_resolvents_are_bounded`).
- kissat (`gates.c`, `kissat_find_gates`) tries equivalence (`equivalences.c`), AND in both polarities (`ands.c`), if-then-else (`ifthenelse.c`) and finally a general definition (`definition.c`). There is no XOR pattern in kissat's elimination; XORs are used in congruence closure (chapter 5).
- MiniSat 2.2.0 does no gate detection.

kissat's `definition.c` finds definitions of any shape with a trick: remove x and $\overline{x}$ from all clauses containing x and hand the result to the small embedded solver `kitten`. If that is unsatisfiable, the clauses in the unsatisfiable core are a definition of x. In the example: removing x leaves $(a), (b), \left(\overline{a} \vee \overline{b}\right)$ from the gate and $(c), (f), (d), (e)$ from the rest. The first three are already contradictory, so they form the core, and kitten has rediscovered the AND gate without knowing what an AND gate is. For such general definitions kissat may also need gate-with-gate resolvents (the `resolve_gate` flag in `resolve.c`).

CaDiCaL: `gates.cpp` (used by `elim.cpp`) · kissat: `gates.c`, `ands.c`, `equivalences.c`, `ifthenelse.c`, `definition.c` (with `kitten.c`) · MiniSat: — · ksat port: not ported

Blocked clause elimination

A clause C is **blocked** on one of its literals l if every resolvent of C on l (with all clauses containing $\bar{l}$) is tautological. Such a clause can be deleted and the formula stays equisatisfiable. Intuition: the clauses with $\bar{l}$ do not care whether l is true, because each of them is already satisfied by some other literal whenever C is falsified apart from l. So if a model falsifies C, flipping l to true repairs C without breaking anything.

A pure literal (no clause contains $\bar{l}$) is the extreme case: every clause containing l is blocked, since there is nothing to resolve with.

> **A blocked clause and its repair**
> $C_1 = \left(a \vee \bar{b}\right)$, $C_2 = (\bar{a} \vee b)$, $C_3 = (a \vee c)$.
>
> Is C_1 blocked on a? The only clause with $\bar{a}$ is C_2, and the resolvent is $\left(\bar{b} \vee b\right)$, tautological. Yes. Delete C_1.
>
> The reduced formula C_2, C_3 has the model $a = 0, b = 1, c = 1$, which falsifies C_1. So the solver must remember "C_1, witness a". At the end: C_1 is false, flip a to 1. Now $a = 1, b = 1, c = 1$: C_1 true (via a), C_2 true (via b), C_3 true.

Why it helps: blocked clauses are typically redundant pieces of encodings (for example one direction of a definition whose other direction is all that is needed). Removing them makes other variables easier to eliminate. BCE itself only deletes clauses (although deleting all clauses of a pure literal leaves its variable unused).

CaDiCaL runs `block ()` inside `elim ()`, just before the elimination rounds, and only tries literals whose negation recently disappeared from an irredundant clause (header comment of `block.cpp`). It has special fast paths for pure literals and literals with one negative occurrence. The option `block` is off by default in CaDiCaL 2.1.3. kissat and MiniSat do not have BCE.

CaDiCaL: `block.cpp` (option `block`, default off) · kissat: — · MiniSat: — · ksat port: not ported

Covered clause elimination

Covered clause elimination (CCE) is a stronger version of BCE. Before checking whether C is blocked, it first **extends** C with extra literals that can be added without changing satisfiability, which makes it easier for the extended clause to become blocked (or tautological). Two ways to add literals:

- **Covered literal addition** (CLA): look at all clauses with $\bar{l}$ for some l in C whose resolvent with C is not tautological. If they all share some literal m (besides $\bar{l}$), add m to C. Intuition: every way of using $\bar{l}$ already needs m, so C may as well "cover" it.
- **Asymmetric literal addition** (ALA): if the other clauses together with all literals of C being false force some literal m to be false, add m to C. This is unit propagation in disguise.

If the extended clause ends up blocked (or propagation hits a conflict), the original C is removed.

> **A covered clause**
> $C_1 = (a \vee b)$, $C_2 = (\bar{a} \vee c \vee d)$, $C_3 = \left(\bar{a} \vee c \vee \bar{d}\right)$, $C_4 = \left(\bar{c} \vee \bar{b}\right)$.
>
> C_1 is not blocked: on a the resolvent with C_2 is $(b \vee c \vee d)$, on b the resolvent with C_4 is $(a \vee \bar{c})$.

CLA on a: the clauses with $\overline{a}$ are C_2, C_3; both contain c (and d appears with different signs). Add c: $C_{1'} = (a \vee b \vee c)$.

Is $C_{1'}$ blocked on c? The only clause with $\overline{c}$ is C_4, and the resolvent $\left(a \vee b \vee \overline{b}\right)$ is tautological. Yes. So C_1 is deleted.

Two steps go on the reconstruction stack: first "$(a \vee b)$, witness a" (the CLA step), then "$(a \vee b \vee c)$, witness c". At the end they are undone newest first. The reduced formula C_2, C_3, C_4 has the model $a = b = c = d = 0$. Newest entry: $(a \vee b \vee c)$ is false, set $c = 1$. Next: $(a \vee b)$ is false, set $a = 1$. Result $a = 1, b = 0, c = 1, d = 0$, a model of all four clauses.

Note that $(a \vee b)$ and $(a \vee b \vee c)$ are not interchangeable in the formula: with C_2, C_3, C_4 the assignment $a = 0, b = 0, c = 1$ satisfies the second but not the first. CLA only preserves satisfiability, which is exactly why it needs reconstruction entries.

CaDiCaL implements the asymmetric variant (ALA plus CLA) in `cover.cpp` and runs `cover()` after `block()` inside `elim()`. It refuses CLA on frozen literals (`cover_propagate_covered`). The option `cover` is off by default in 2.1.3.

CaDiCaL: `cover.cpp` (option `cover`, default off) · kissat: — · MiniSat: — · ksat port: not ported

Globally blocked clauses (conditioning)

For a blocked clause, the repair flips one literal of the clause. **Globally blocked** clauses allow the repair to flip several literals, including variables that do not even occur in the clause. CaDiCaL calls this *conditioning* (`condition.cpp`, after Kiesl, Heule and Biere).

The idea uses an **autarky**: an assignment ω that satisfies every clause it touches. Setting an autarky can never hurt. Conditioning looks for a *conditional* autarky: a partial assignment α (the condition) and an assignment ω such that every clause touched by ω is satisfied either by ω or by α. If α consists of negations of literals of C, and ω satisfies some literal of C, then C can be removed. Repair at the end: if C is false, set all literals of ω.

A clause repaired by flipping two variables

$C_1 = (a \vee b)$, $C_2 = (\overline{a} \vee c)$, $C_3 = \left(\overline{b} \vee \overline{c}\right)$, $C_4 = (b \vee d)$.

Take $\alpha = \{b = 0\}$ (the negation of the literal b of C_1) and $\omega = \{a = 1, c = 1\}$. Check: ω satisfies a in C_1. C_2 is touched by ω and satisfied by $c = 1$. C_3 is touched by ω (via c) and not satisfied by ω, but it is satisfied by α ($\overline{b}$). C_4 is not touched by ω. So C_1 is globally blocked and is removed, with witness $\{a, c\}$.

The reduced formula has the model $a = 0, b = 0, c = 0, d = 1$, which falsifies C_1. Flipping only a would break C_2. Flipping the whole witness gives $a = 1, b = 0, c = 1, d = 1$, a model of all four clauses.

(This particular C_1 is also caught by CCE: CLA on a adds c, and $(a \vee b \vee c)$ is blocked on c. The example is chosen small, not to show a case only conditioning finds.)

CaDiCaL does not search for α and ω from scratch. It uses the current partial assignment of the CDCL search, splits it into a conditional part and an autarky part, and then refines that split for each candidate clause. It is triggered during search when the decision level is above the average backjump level. This was the first technique in CaDiCaL that needs witnesses with more than one literal on the reconstruction stack. The option `condition` is off by default in 2.1.3.

CaDiCaL: `condition.cpp` (option `condition`, default off) · kissat: — · MiniSat: — · ksat port: not ported

Instantiation

Instantiation shortens a clause instead of deleting it. For a literal l in clause C: assume l true and every other literal of C false, then run unit propagation. If that leads to a conflict, then l can be removed from C. Reason: the conflict shows that the formula implies "$\bar{l}$ or one of the other literals of C", and resolving that with C itself on l gives C without l.

> **Removing a literal by instantiation**
> $C_1 = (a \vee b \vee c)$, $C_2 = (\bar{a} \vee d)$, $C_3 = \left(\bar{a} \vee b \vee \bar{d}\right)$.
>
> Try a in C_1: assume $a = 1, b = 0, c = 0$. C_2 forces $d = 1$, then C_3 has all literals false: conflict. So C_1 becomes $(b \vee c)$, and the formula is equivalent to the original.
>
> Now a only occurs as $\bar{a}$, so a is a pure literal and the next elimination round removes it for free.

That last effect is the point of CaDiCaL's version (`instantiate.cpp`): candidates are literals with few occurrences, on variables where elimination just failed, collected at the end of an elimination round (`elim_round` in `elim.cpp` calls `collect_instantiation_candidates`). Removing a rare literal can turn a variable pure. The clause must have at least three unassigned literals. The option `instantiate` is off by default in 2.1.3.

CaDiCaL: `instantiate.cpp` (option `instantiate`, default off) · kissat: — · MiniSat: — · ksat port: not ported

Bounded variable addition (factorization)

This is elimination run backwards: introduce a new variable if that makes the formula smaller. If the clauses form a "product", every literal from a set L combined with every literal from a set K, then $|L| \cdot |K|$ clauses can be replaced by $|L| + |K|$ clauses using one fresh variable x.

> **Factoring nine clauses into six**
> The nine clauses $(a \vee d), (a \vee e), (a \vee f), (b \vee d), ..., (c \vee f)$ say "a, b, c all true, or d, e, f all true". With a fresh x:
>
> $$(a \vee x), (b \vee x), (c \vee x), (\overline{x} \vee d), (\overline{x} \vee e), (\overline{x} \vee f)$$
>
> If $x = 0$ the first three force a, b, c; if $x = 1$ the last three force d, e, f. The models restricted to $a...f$ are exactly the original ones. 9 clauses become 6.

Notice that eliminating x from the six clauses produces exactly the nine clauses again (9 resolvents for 6 clauses). That is why kissat only factors when the reduction is larger than the current elimination bound (`reduction > factoring.bound` in `run_factorization`, where the bound is the elimination's `additional_clauses`); otherwise BVE would immediately undo it.

kissat's `factor.c` runs during probing (`probe.c`, also in the initial preprocessing). It searches for such products among clauses up to size `factorsize` (5), greedily picking the factor that removes the most clauses (`best_quotient`). Fresh variables need no reconstruction: any model of the new formula, with x ignored, is a model of the old one. CaDiCaL 2.1.3 and MiniSat 2.2.0 do not have factorization.

CaDiCaL: — · kissat: `factor.c` · MiniSat: — · ksat port: not ported

Model reconstruction: the extension stack

Everything that only preserves satisfiability (BVE, BCE, CCE, conditioning) leaves the solver with a model of the reduced formula that may violate removed clauses, or may not even assign eliminated variables. The fix, pioneered in MiniSat, is an **extension stack** (MiniSat `elimclauses`, CaDiCaL `extension`, kissat `extend`; also called the *witness stack*). Each removed clause is pushed together with its *witness*: the literal(s) that repair it.

After the search finds a model of the reduced formula, the stack is walked from the top (newest entry) to the bottom:

- if the clause is satisfied by the current values, do nothing;
- otherwise set every witness literal to true.

Going newest-first matters: a later simplification ran on a formula that already lacked the earlier clauses, so its repairs have to be done before the earlier ones are checked.

> **Extending a model after BVE**
> Original: $C_1 = (x \vee a)$, $C_2 = (\overline{x} \vee b)$, $C_3 = (\overline{x} \vee c)$. After eliminating x: $(a \vee b), (a \vee c)$.
>
> MiniSat (`eliminateVar`) saves only the smaller side: here the one positive clause $(x \vee a)$ with witness x, and on top a unit $(\overline{x})$ that just sets the default $x = 0$.
>
> The reduced formula has the model $a = 0, b = 1, c = 1$; x has no value. Walk the stack from the top:
> 1. unit $(\overline{x})$: set $x = 0$;
> 2. $(x \vee a)$: both false, so set the witness $x = 1$.
>
> Result $x = 1, a = 0, b = 1, c = 1$: C_1 via x, C_2 via b, C_3 via c. For the other model $a = 1, b = 0, c = 0$, step 2 finds $(x \vee a)$ satisfied and $x = 0$ stays, which is needed because C_2 would fail with $x = 1$.

Why it is enough to save one side: if all positive clauses are satisfied by the other variables, $x = 0$ satisfies all negative clauses (they contain $\overline{x}$). If some positive clause needs $x = 1$, its other literals are all false, and then each resolvent with a negative clause forces that negative clause's other literals true, so the negative clauses hold with $x = 1$ as well.

The three solvers differ in what they save:

- MiniSat stores the smaller side plus a default unit (`mkElimClause`), and `extendModel` walks `elimclauses` from the end.
- kissat also stores the smaller side plus one unit for the other side (the smaller side is picked in `kissat_generate_resolvents`, the unit is added in `weaken_clauses` in `eliminate.c` unless the `incremental` option is set), and `extend.c` walks `extend` from the end.
- CaDiCaL stores all clauses of both sides (`mark_eliminated_clauses_as_garbage` in `elim.cpp`). The comment there says why: the one-side trick would break its incremental `restore` logic (next section). The exception is elimination with a gate (section "Gates and definitions"): then only the gate clauses are stored, since the gate alone defines x (the test `if (!substitute || c->gate)` there). `External::extend` in `extend.cpp` walks the stack from the end; witnesses may have several literals (needed for conditioning).

CaDiCaL: `extend.cpp` (`External::extend`, `push_clause_on_extension_stack`) · kissat: `extend.c`, `weaken.c` · MiniSat: `SimpSolver.cc` (`mkElimClause`, `extendModel`) · ksat port: not needed, nothing is eliminated

Frozen variables

Elimination assumes the formula is final. In incremental use (chapter 6) it is not: the user may add clauses or *assume* literals in the next `solve` call, and an IPASIR-UP propagator watches its *observed*

variables and expects to see their assignments. If such a variable was eliminated, the solver gives wrong answers.

> **A spurious model after elimination**
> As before, x is eliminated from $(x \vee a), (\overline{x} \vee b), (\overline{x} \vee c)$, leaving $(a \vee b), (a \vee c)$.
>
> Next call, the user assumes $x = 0$ and $a = 0$. With the original clauses this is unsatisfiable: $x = 0$ and $a = 0$ falsify $(x \vee a)$. But the reduced formula does not mention x any more, so it happily reports $a = 0, b = 1, c = 1$ (with $x = 0$). And the reconstruction step would then flip x to 1, contradicting the assumption.

A **frozen** variable is one that elimination (and similar techniques) must not touch. The three solvers handle this differently:

- MiniSat: the user calls `setFrozen(v, true)` for variables used later. `solve_` also freezes the current assumptions temporarily before calling `eliminate()` and unfreezes them afterwards. `eliminateVar` asserts the variable is not frozen.
- CaDiCaL: has `freeze` / `melt` with reference counts, but its header (`cadical.hpp`) calls this the old way. By default it instead **restores**: if a later `add` or `assume` uses a literal whose negation is a witness on the extension stack, `restore.cpp` puts the removed clauses back (recursively) before solving. Observed variables of an IPASIR-UP propagator are frozen when they are added (`add_observed_var` in `external.cpp` calls `freeze`), and `cover.cpp` skips frozen literals.
- kissat: upstream is not incremental (a second `kissat_solve` is rejected in `internal.c`: "incremental solving not supported"), so it has no freeze API.

> **ksat ports**
> None of this chapter is ported: the ksat Kotlin ports of MiniSat, CaDiCaL and kissat have no pre- or inprocessing, and MiniSat's `SimpSolver` layer is not ported. Every variable stays in the formula, so assumptions and IPASIR-UP observed variables are always safe and there is nothing to freeze or restore. Freezing becomes relevant the moment any elimination technique is ported: assumptions and observed variables must then be frozen or restored, exactly as upstream does.

CaDiCaL: `external.cpp` (`freeze`, `melt`), `restore.cpp`, `cadical.hpp` · kissat: — · MiniSat: `SimpSolver.cc` (`setFrozen`, `solve_`) · ksat port: not needed (no elimination)

Simplifying the formula II: probing, equivalences, strengthening

Chapter 4 removed things from the formula: subsumed clauses, eliminated variables, blocked clauses. This chapter is about the opposite direction. The solver spends a little time **deriving** new short facts that the formula implies but does not state explicitly: units ("a must be false"), binary clauses ("if a then d"), and equivalences ("x and y always have the same value"). Short facts are valuable because unit propagation uses them immediately, and an equivalence lets the solver delete a whole variable.

Almost every technique here is built from one cheap tool you already know from chapter 1: unit propagation. The solver assigns a literal on purpose, propagates, and reads off what happened. A few techniques also use the structure of the binary clauses (the **binary implication graph**), and the last one, SAT sweeping, runs a tiny second SAT solver.

All of this is **inprocessing**: it runs at decision level 0, between phases of the normal CDCL search, under an effort budget that is a fraction of the search effort. Nothing has to run to completion; whatever was found is kept, the rest is tried next time.

Where these run in the two solvers

CaDiCaL 2.1.3 groups them into two phases. Its `probe()` (in `probe.cpp`) runs equivalent literal substitution (`decompose`), then ternary resolution, then failed literal probing, then `decompose` again. Vivification and transitive reduction are scheduled at the end of `subsume()` (in `subsume.cpp`).

kissat 4.0.4 has a single "probe" phase (`probe.c`) that calls, in this order: congruence closure, substitution, binary backbone, vivification, sweeping, substitution, transitive reduction, binary backbone, and factoring. kissat 4.0.4 has no classic failed literal probing and no ternary resolution; the binary backbone takes over the role of the former.

The binary implication graph

Binary clauses are special. A binary clause $(x \vee y)$ says "if x is false then y is true, and if y is false then x is true". So each binary clause gives two directed edges between literals:

$$(x \vee y) \quad \Rightarrow \quad \overline{x} \to y \quad \text{and} \quad \overline{y} \to x.$$

The graph over all $2n$ literals with these edges is the **binary implication graph** (BIG). Following edges from a literal is exactly unit propagation restricted to binary clauses. The graph is symmetric in a twisted way: whenever there is an edge $u \to v$ there is also the "contrapositive" edge $\overline{v} \to \overline{u}$. So every path has a mirror path with all literals negated and the direction reversed.

Reading the graph off three clauses

$C_1 = (\overline{a} \vee b)$, $C_2 = \left(\overline{b} \vee c\right)$, $C_3 = (\overline{c} \vee a)$ give the edges

clause	edge	mirror edge
$C_1 = (\overline{a} \vee b)$	$a \to b$	$\overline{b} \to \overline{a}$
$C_2 = \left(\overline{b} \vee c\right)$	$b \to c$	$\overline{c} \to \overline{b}$
$C_3 = (\overline{c} \vee a)$	$c \to a$	$\overline{a} \to \overline{c}$

There is a cycle $a \to b \to c \to a$ and its mirror $\overline{a} \to \overline{c} \to \overline{b} \to \overline{a}$.

Two words from graph theory are used in the next section. A *root* is a literal with outgoing edges but no incoming edge: nothing implies it through a binary clause. In the example above there is no root, since every literal lies on a cycle. Seen from a start literal s, a literal d *dominates* a literal l if every path from s to l passes through d; so s itself dominates everything it reaches. When unit propagation starts from s and follows only binary clauses, each implied literal records the one literal that implied it, and these links form a tree with s at the top. In that tree the dominators of l are simply the literals on the path from l up to s.

The BIG is the data structure behind failed literal probing, equivalent literal substitution and transitive reduction. Solvers do not build the graph explicitly; the watch lists of binary clauses already are its adjacency lists.

CaDiCaL: used in `probe.cpp`, `decompose.cpp`, `transred.cpp` · kissat: used in `substitute.c`, `transitive.c`, `backbone.c` · MiniSat: — · ksat port: not ported

Failed literal probing

Intuition. Pick a literal, say a, and pretend it is true: put it on the trail at decision level 1 and run unit propagation. If propagation ends in a conflict, then $a = 1$ is impossible in every model. So $\overline{a}$ is a unit, and it can be added at level 0 permanently. The probe "failed", hence the name.

If the probe does not fail, nothing is learned (in the basic version) and the solver simply backtracks to level 0 and tries the next literal.

A failed literal
Formula: $C_1 = (\overline{a} \vee b)$, $C_2 = (\overline{a} \vee c)$, $C_3 = \left(\overline{b} \vee \overline{c} \vee d\right)$, $C_4 = \left(\overline{b} \vee \overline{d}\right)$, $C_5 = (a \vee e \vee f)$.

Probe a:
- $a@1$ (the probe, a decision on level 1)
- $b@1 \leftarrow C_1$
- $c@1 \leftarrow C_2$
- $d@1 \leftarrow C_3$ (both $\overline{b}$ and $\overline{c}$ are false)
- $C_4 = \left(\overline{b} \vee \overline{d}\right)$ is now fully false: conflict.

So $a = 1$ leads to a contradiction, and the solver adds the unit $\overline{a}$. Back at level 0 it propagates $\overline{a}$: C_1, C_2 are satisfied, C_5 shrinks to $(e \vee f)$. Probing $\overline{a}$ instead would not have failed; only one polarity gives a unit here.

A stronger unit than the probe. The conflict often does not depend on the probe itself but on some literal implied by it. If the probe a implies b through a binary clause, and the conflict only needs b, then $\overline{b}$ is a unit as well, and it is stronger: it also gives $\overline{a}$ by propagation. CaDiCaL computes this "probing UIP" in `failed_literal()` as the closest common dominator of the conflict literals in the tree of binary implications from the previous section (function `probe_dominator()`), learns its negation as a unit, and then also the negations of the literals on the path back up to the probe.

Which literals to probe. Probing every literal is expensive: one full propagation per probe. CaDiCaL only probes *roots* of the binary implication graph (previous section): literals l such that $\overline{l}$ occurs in some binary clause but l itself does not. In the example a is such a root, since the binary clauses contain only $\overline{a}$. The reasoning: if l has a predecessor k (a binary clause gives $k \to l$), probing k propagates everything that probing l would, and more. CaDiCaL also skips a probe if no new unit was found since the last time that literal was probed (the `propfixed` counter).

Hyper-binary resolution

While probing a, every literal assigned at level 1 is implied by a. So for each such literal l the binary clause $(\overline{a} \vee l)$ is implied by the formula. Adding all of them would be far too many, but there is one useful case: when l was forced by a **long** clause (three or more literals). Then the solver adds a binary clause that, next time, gets the same implication with a single binary propagation instead of a long-clause visit. This is *hyper-binary resolution* (HBR): it resolves the long reason clause with several binary clauses in one step ("hyper").

Hyper-binary resolvent while probing
Formula: $C_1 = (\overline{a} \vee b)$, $C_2 = (\overline{a} \vee c)$, $C_3 = \left(\overline{b} \vee \overline{c} \vee d\right)$, $C_4 = \left(a \vee \overline{d} \vee e\right)$.

Probe a: $a@1$, $b@1 \leftarrow C_1$, $c@1 \leftarrow C_2$, $d@1 \leftarrow C_3$. No conflict.

d was forced by the ternary clause C_3. Its false literals are $\overline{b}$ and $\overline{c}$; both b and c were implied by a through binary clauses, so a is their common dominator. The hyper-binary resolvent is $(\overline{a} \vee d)$: resolve C_3 with C_1 on b, then with C_2 on c. It is implied by the formula, and from now on $a = 1$ gives $d = 1$ through one binary clause.

CaDiCaL does this on the fly in `hyper_binary_resolve()` during probing on level 1. The new clause consists of the newly implied literal and the negated dominator of all the false literals in the reason clause. If the resolvent happens to subsume the reason (the negated dominator was already in the reason clause), CaDiCaL adds the binary clause and marks the long clause as garbage. Most hyper-

binary resolvents are learned (redundant) clauses, so the clause database can drop them again. Transitive reduction (below) exists partly to keep their number under control.

Just before the probing rounds, CaDiCaL also removes duplicated binary clauses and does **hyper unary resolution** in the same pass: from $(a \vee b)$ and $(\overline{a} \vee b)$ it derives the unit b (`mark_duplicated_binary_clauses_as_garbage()`).

Why it helps, and the cost. Every unit removes a variable from the problem for good, and the CDCL search would otherwise have to rediscover the same conflict over and over below different decisions. The cost is one propagation per probe, which is why CaDiCaL limits the total probing propagations to a fraction of the search propagations (options `probereleff`, `probemineff`, `probemaxeff`) and keeps untried probes for the next call.

CaDiCaL: `probe.cpp` (`probe_round`, `failed_literal`, `probe_dominator`, `hyper_binary_resolve`; option `probehbr`) · kissat: no classic probing in 4.0.4, see "Backbone computation" (`backbone.c`) · MiniSat: — · ksat port: not ported

Vivification

Intuition. Vivification takes one existing clause and asks: "is a shorter version of this clause already implied by the other clauses?" It tests this with unit propagation. For a clause $C = (l_1 \vee l_2 \vee ... \vee l_k)$ the solver assumes the literals are false one after the other, $\overline{l_1}$, $\overline{l_2}$, …, and propagates the *other* clauses after each step. Three things can happen:

1. Propagation makes some later literal l_j of C **false**. Then l_j is useless in C and can be dropped.
2. Propagation runs into a **conflict** after assuming $\overline{l_1}, ..., \overline{l_i}$. Then the prefix $(l_1 \vee ... \vee l_i)$ is implied by the other clauses, and C can be shortened to it (in practice to the literals that conflict analysis found responsible).
3. Propagation makes some later literal l_j **true**. Then C is implied by the other clauses as a whole (an "asymmetric tautology"), and an irredundant C can even be deleted.

The idea goes back to Piette, Hamadi and Sais (2008); the CaDiCaL comment calls it a special case of asymmetric tautology and asymmetric literal elimination.

> **Vivifying a four-literal clause**
> Clause to vivify: $C_1 = (a \vee b \vee c \vee d)$. Other clauses: $C_2 = (a \vee e)$, $C_3 = (\overline{e} \vee \overline{c})$, $C_4 = (b \vee \overline{e} \vee f)$, $C_5 = \left(b \vee \overline{e} \vee \overline{f}\right)$.
>
> - Assume $\overline{a}$: $e \leftarrow C_2$, then $\overline{c} \leftarrow C_3$. The literal c of C_1 is now false (case 1), so c could already be dropped.
> - Assume $\overline{b}$: $f \leftarrow C_4$, and $C_5 = \left(b \vee \overline{e} \vee \overline{f}\right)$ is fully false: conflict (case 2).
>
> So $(a \vee b)$ is implied by $C_2, ..., C_5$ alone, and C_1 is replaced by $(a \vee b)$. The shorter clause subsumes the old one, so the set of models does not change.

Note the order: once the conflict appears after two literals, the rest of C_1 (c and d) is cut off, which already covers the dropping of c from step 1.

Details in the real solvers. Both solvers vivify learned (redundant) clauses as well as irredundant ones. kissat sorts the learned clauses into tiers by glue (chapter 3) and vivifies each tier separately; CaDiCaL 2.1.3 runs one round over the irredundant clauses and one over all clauses. CaDiCaL sorts the literals of all candidates by how often they occur and sorts the candidates so that consecutive clauses share prefixes. Then it can *reuse* the decisions of the previous candidate instead of starting over (the idea of "distillation" from Han and Somenzi, done without an explicit trie). It also runs conflict analysis to find which of the assumed literals were really needed, and it can *instantiate* the last literal (option `vivifyinst`). Case 3 (delete the clause because it is implied) is only allowed when propagation

used irredundant clauses only; a learned clause may be deleted later, so it cannot justify removing an original clause.

Why it helps, and the cost. Shorter clauses propagate earlier, and learned clauses in particular are often much longer than they need to be. In modern solvers vivification is one of the most effective inprocessing steps. Its cost is a propagation per tested literal, so it is budgeted relative to search propagations (`vivifyreleff` in CaDiCaL, `vivifyeffort` in kissat).

CaDiCaL: `vivify.cpp` (called from `subsume.cpp`) · kissat: `vivify.c` · MiniSat: — · ksat port: not ported

Equivalent literal substitution

Intuition. If the BIG has a cycle $a \to b \to c \to a$, then a, b and c imply each other, so they are equal in every model. Literals that can all reach each other form a *strongly connected component* (SCC). All literals in one SCC are equivalent. The solver picks one *representative* per SCC, replaces every other literal of the SCC by it in all clauses, and the replaced variables disappear from the formula. This is *equivalent literal substitution* (ELS), also called equivalence reasoning or "decompose".

Two side effects come for free:

- If a literal and its own negation end up in the same SCC ($x \to \ldots \to \overline{x} \to \ldots \to x$), the formula is unsatisfiable.
- After substitution some clauses contain both l and $\overline{l}$ (tautologies, deleted), some contain the same literal twice (shortened), and some become duplicates.

SCCs are found in linear time with Tarjan's algorithm, one depth-first search over the BIG. By the mirror property, the SCC of $\overline{a}, \overline{b}, \overline{c}$ is the mirror of the SCC of a, b, c and gets the mirrored representative.

> **Substituting an SCC**
> $C_1 = (\overline{a} \vee b)$, $C_2 = \left(\overline{b} \vee c\right)$, $C_3 = (\overline{c} \vee a)$ (the cycle from the example in "The binary implication graph"), plus $C_4 = (b \vee d \vee e)$, $C_5 = \left(\overline{c} \vee \overline{d}\right)$, $C_6 = (b \vee \overline{c} \vee e)$.
>
> SCCs: $\{a, b, c\}$ and its mirror $\left\{\overline{a}, \overline{b}, \overline{c}\right\}$. Choose a as the representative: replace b by a and c by a (and $\overline{b}, \overline{c}$ by $\overline{a}$).
> - C_1, C_2, C_3 become $(\overline{a} \vee a)$: tautologies, deleted.
> - C_4 becomes $(a \vee d \vee e)$, C_5 becomes $\left(\overline{a} \vee \overline{d}\right)$.
> - C_6 becomes $(a \vee \overline{a} \vee e)$: tautology, deleted.
>
> Result: two clauses over a, d, e instead of six over $a, \ldots, e$. Every model of the new formula extends to a model of the old one by setting $b = a$ and $c = a$, and every model of the old formula arises that way.

The last sentence is model reconstruction from chapter 4: the solver keeps the equivalences $b \equiv a$ and $c \equiv a$ (as the two binary clauses each) on its reconstruction stack, so the final model can assign the removed variables. CaDiCaL pushes them in `decompose.cpp` onto the extension stack; kissat's `substitute.c` does the same through `weaken.c`.

Why it helps, and the cost. Each non-trivial SCC removes variables, merges clauses, and often exposes new subsumptions. It is cheap (linear in the number of binary clauses), so both solvers run it several times per phase: CaDiCaL before and after probing and again after ternary resolution if that produced binary clauses; kissat twice per probe phase and after congruence closure and sweeping, which feed it new equivalences.

CaDiCaL: `decompose.cpp` (option `decompose`) · kissat: `substitute.c` (option `substitute`) · MiniSat: — · ksat port: not ported

Transitive reduction

Intuition. If the BIG has edges $a \to b$ and $b \to c$, then the direct edge $a \to c$ adds nothing: propagation reaches c from a anyway via b. The binary clause behind such a shortcut edge is *transitive* and can be deleted. Removing all of them is the *transitive reduction* of the graph.

> **A transitive binary clause**
> $C_1 = (\overline{a} \vee b)$, $C_2 = \left(\overline{b} \vee c\right)$, $C_3 = (\overline{a} \vee c)$.
>
> Edges $a \to b$, $b \to c$, $a \to c$. Without C_3 there is still the path $a \to b \to c$. Indeed C_3 is the resolvent of C_1 and C_2 on b, so the formula $C_1 \wedge C_2$ already implies it, and C_3 is removed.

How it is computed. For each candidate binary clause, say with edge $s \to t$, the solver searches the BIG from s (a breadth-first propagation over binary clauses) while ignoring this one clause. If it reaches t, the clause is transitive and is deleted. If the search instead reaches both some literal and its negation, then s is a failed literal and $\overline{s}$ becomes a unit. So transitive reduction finds some failed literals as a by-product. An irredundant clause may only be removed using a path of irredundant clauses (a learned clause along the path might be deleted later). CaDiCaL does not try hyper-binary resolvents as candidates, since they come in large numbers and are usually not transitive when added.

Why it helps, and the cost. Fewer binary clauses mean shorter watch lists and fewer redundant propagation steps. CaDiCaL's source says it plainly: this matters mostly because hyper-binary resolution would otherwise produce too many binary clauses. A full reduction is too expensive for large formulas, so it is bounded like probing and resumes next time.

CaDiCaL: `transred.cpp` (option `transred`, called from `subsume.cpp`) · kissat: `transitive.c` (option `transitive`) · MiniSat: — · ksat port: not ported

Ternary resolution

Intuition. Bounded variable elimination (chapter 4) adds resolvents only to remove a variable. Ternary resolution adds resolvents for their own sake, but only small ones: it resolves pairs of *ternary* clauses (three literals) and keeps the resolvent only if it has at most three literals and is not already present or subsumed by a binary clause. Resolvents with four literals are dropped. A binary resolvent is the jackpot: it subsumes both ternary clauses it came from, so they are deleted, and it adds new edges to the BIG.

The interesting use is with **gates** (chapter 4). If the formula encodes the same XOR or if-then-else twice, with two different output variables, ternary resolution plus ELS can discover that the outputs are equivalent.

> **Two copies of the same XOR**
> Let $c = \mathrm{XOR}(a, b)$ and $d = \mathrm{XOR}(a, b)$, each with its usual four ternary clauses: $(a \vee b \vee \overline{c})$, $\left(\overline{a} \vee \overline{b} \vee \overline{c}\right)$, $(\overline{a} \vee b \vee c)$, $\left(a \vee \overline{b} \vee c\right)$, and the same four with d in place of c.
>
> Probing does not help: assuming c (or $\overline{c}$, d, $\overline{d}$) propagates nothing, because every clause still has two open literals. Ternary resolution:
> - $(a \vee b \vee \overline{c})$ and $(\overline{a} \vee b \vee d)$ on a: $R_1 = (b \vee \overline{c} \vee d)$.
> - $\left(\overline{a} \vee \overline{b} \vee \overline{c}\right)$ and $\left(a \vee \overline{b} \vee d\right)$ on a: $R_2 = \left(\overline{b} \vee \overline{c} \vee d\right)$.
> - R_1 and R_2 on b: $(\overline{c} \vee d)$, a binary clause.
>
> Symmetrically one gets $\left(c \vee \overline{d}\right)$. Now the BIG has $c \to d$ and $d \to c$, and equivalent literal substitution merges d into c.

The CaDiCaL source describes this as simulating "structural hashing" of multiplexer (ITE) and binary XOR gates when combined with `decompose`. That is why `probe()` calls `decompose()` again whenever `ternary()` produced a binary clause. The resolvents themselves become clauses of the formula, so later resolution steps (in the same round or the next of `ternaryrounds`) can use them, as R_1 and R_2 were used above.

Why it helps, and the cost. It finds binary clauses and equivalences that unit propagation cannot see, at bounded cost: CaDiCaL counts steps like propagations, limits the number of added clauses (`ternarymaxadd`), and skips literals with too many occurrences (`ternaryocclim`). kissat 4.0.4 does not have ternary resolution; congruence closure (below) finds gate equivalences like the XOR one directly.

CaDiCaL: `ternary.cpp` (option `ternary`) · kissat: — (not in 4.0.4) · MiniSat: — · ksat port: not ported

Backbone computation

Intuition. The *backbone* of a satisfiable formula is the set of literals that are true in every model. A backbone literal can be added as a unit without losing any model. The exact test is simple but expensive: l is a backbone literal if and only if the formula plus the unit $\overline{l}$ is unsatisfiable, which is a full SAT call per literal.

Failed literal probing is a cheap under-approximation: if probing $\overline{l}$ fails, then l is in the backbone. kissat makes this systematic for the binary clauses: `backbone.c` computes backbone literals of the **binary part** of the formula. For each candidate literal it assigns it as a decision and propagates over binary clauses only. A conflict means the candidate failed; kissat analyzes the conflict to find the UIP (the same dominator idea as in CaDiCaL's probing) and adds its negation as a unit. Since the binary clauses are part of the formula, every literal true in all models of the binary part is also true in all models of the full formula.

> **Backbone of a small formula**
> $C_1 = (\overline{a} \vee b)$, $C_2 = (\overline{a} \vee c)$, $C_3 = \left(\overline{b} \vee \overline{c}\right)$, $C_4 = (a \vee d)$, $C_5 = \left(\overline{d} \vee e \vee f\right)$, $C_6 = \left(\overline{e} \vee \overline{f}\right)$.
>
> Candidate a, propagating binary clauses only: $a@1$, $b@1 \leftarrow C_1$, $c@1 \leftarrow C_2$, and C_3 is false: conflict. The UIP is a itself, so $\overline{a}$ is a unit. Propagating it at level 0 gives $d \leftarrow C_4$.
>
> The backbone of this formula is exactly $\{\overline{a}, d\}$: in every model $a = 0$ and $d = 1$, while each of b, c, e, f takes both values in some models. Here the binary clauses alone already have the same backbone.

kissat's other source of backbone literals is SAT sweeping (last section): there the small sub-solver kitten tests backbone candidates exactly, but only on a local part of the formula.

Why it helps, and the cost. Units are the best simplification there is. Restricting the search to binary clauses makes each probe very cheap, and kissat's version is bounded by "ticks" (`backboneeffort`) and a round limit. CaDiCaL 2.1.3 has no separate backbone procedure; its failed literal probing covers similar ground.

CaDiCaL: — (failed literal probing in `probe.cpp` is the closest) · kissat: `backbone.c` (option `backbone`, "binary clause backbone") · MiniSat: — · ksat port: not ported

Congruence closure

Intuition. Many CNF formulas are translated from circuits, so they contain *gates*: groups of clauses that define one output variable as AND, XOR or if-then-else of some inputs (chapter 4 shows how gates are detected for variable elimination). If two gates have the same type and the same inputs, their

outputs must be equal. In a circuit tool this is called structural hashing; in logic it is *congruence*: equal arguments give equal function values.

The "closure" part is what makes it strong. Once two outputs are merged, gates that use them as inputs may suddenly have identical inputs too, so their outputs can be merged as well, and so on until nothing changes.

> **Merging gates, then merging again**
> Four AND gates, each with its three clauses (for $g = \mathrm{AND}(a, b)$: $(\overline{g} \vee a)$, $(\overline{g} \vee b)$, $\left(g \vee \overline{a} \vee \overline{b}\right)$):
>
> $$g = \mathrm{AND}(a, b), \quad h = \mathrm{AND}(a, b), \quad p = \mathrm{AND}(g, c), \quad q = \mathrm{AND}(h, c).$$
>
> - g and h are the same gate on the same inputs: merge, $h \equiv g$.
> - Rewrite q with the merged input: $q = \mathrm{AND}(g, c)$, which is now the same gate as p: merge, $q \equiv p$.
>
> Two variables disappear. In every model of the twelve clauses $g = h$ and $p = q$.

For AND gates, failed literal probing with hyper-binary resolution could eventually find $g \to h$ as well. For XOR gates it cannot: with $g = \mathrm{XOR}(a, b)$ and $h = \mathrm{XOR}(a, b)$, assuming g, $\overline{g}$, h or $\overline{h}$ propagates nothing (the same situation as in the ternary resolution example). Congruence closure sees the equal gates directly, by hashing.

kissat's `congruence.c` extracts AND gates (`congruenceands`), XOR gates up to a small arity (`congruencexors`, `congruencexorarity`) and if-then-else gates (`congruenceites`), stores them in a hash table keyed by gate type and inputs, merges outputs on a hash hit, rewrites the gates that use a merged literal, and repeats. It also finds units implied by gates, and at the end performs forward subsumption on the rewritten clauses. The found equivalences are handed to `substitute.c` for the actual replacement in the formula.

Why it helps, and the cost. Formulas from hardware verification (for example miters that compare two versions of a circuit) are full of such duplicate structure, and congruence closure can collapse large parts of them before search starts; kissat runs it in its initial probing as well (`preprocesscongruence`). The cost is gate extraction, a pass over the clauses with hashing. When it keeps finding almost nothing, kissat increases a delay so it runs less often.

CaDiCaL: — (not in 2.1.3) · kissat: `congruence.c` (option `congruence`) · MiniSat: — · ksat port: not ported

SAT sweeping with the kitten sub-solver

Intuition. All techniques so far use propagation or syntax. Some equivalences need real search: you have to try cases to see that x and y are always equal. SAT sweeping does exactly that, but only on a *small piece* of the formula, so each search is tiny. The idea comes from circuit verification, where "sweeping" means finding and merging equivalent nodes. kissat's version works like this, one variable at a time:

1. **Environment.** Start at a variable v and collect the clauses around it: first all clauses containing v (depth 1), then all clauses of the variables in those clauses (depth 2), and so on, up to a depth (`sweepdepth`, default 2) and a size limit (`sweepvars`, `sweepclauses`).
2. **Solve.** Load the environment into *kitten*, a small, simple CDCL solver embedded in kissat (`kitten.c`), and solve it. Since the environment is a subset of the formula, it is satisfiable if the formula is.
3. **Candidates from models.** From the model, every literal that is true is a backbone candidate, and all of them start in one equivalence class (all have the same value, true). Each further model splits the classes: literals with different values in some model cannot be equivalent. kitten gets further models from `kitten_flip_literal` (flip one variable if the model stays satisfying) and from the solve calls in the next step.

4. **Prove.** For a backbone candidate l, solve with the assumption $\overline{l}$: if UNSAT, l is a unit. For an equivalence candidate pair x, y, solve with assumptions $x, \overline{y}$ and then $\overline{x}, y$: if both are UNSAT, the binary clauses $(\overline{x} \vee y)$ and $(x \vee \overline{y})$ are implied. If a call is SAT instead, its model refines the classes further.

Because the environment is part of the formula, anything proven on it holds for the whole formula. The binary clauses go into the main solver, and `substitute.c` then removes the merged variable.

Two different circuits for the same function

The formula computes $\mathrm{XOR}(a, b)$ twice, in two different ways:

- $x = \mathrm{XOR}(a, b)$, as one XOR gate (four ternary clauses);
- $u = \mathrm{AND}\left(a, \overline{b}\right)$, $v = \mathrm{AND}(\overline{a}, b)$, $y = \mathrm{OR}(u, v)$, as three gates with three clauses each.

That is 13 clauses. Congruence closure cannot merge x and y: they are outputs of gates of different types with different inputs. Probing does not help either: assuming x, $\overline{x}$, y or $\overline{y}$ never assigns the other one of x, y and never fails.

Sweeping around a with depth 2: depth 1 adds the clauses containing a (the four XOR clauses and two clauses each of u and v), which brings in b, x, u, v; depth 2 adds their remaining clauses, which brings in y. The environment is all 13 clauses. Suppose kitten produces these models:

a	b	x	u	v	y
1	0	1	1	0	1
0	0	0	0	0	0
0	1	1	0	1	1
1	1	0	0	0	0

After the first model the candidate class is $\left\{a, \overline{b}, x, u, \overline{v}, y\right\}$ (the true literals). The second model splits it into $\left\{\overline{b}, \overline{v}\right\}$ and $\{a, x, u, y\}$. The third splits $\{a, x, u, y\}$ into $\{x, y\}$ and $\{a, u\}$; it leaves $\left\{\overline{b}, \overline{v}\right\}$ together, since both are false in it. The fourth ($a = 1$, $b = 1$) separates a from u and $\overline{b}$ from $\overline{v}$. Only $\{x, y\}$ is left, and no literal was true in all four models, so there is no backbone candidate either.

kitten then solves the environment with assumptions $x, \overline{y}$: UNSAT. And with $\overline{x}, y$: UNSAT. So $x \equiv y$, the clauses $(\overline{x} \vee y)$ and $(x \vee \overline{y})$ are added, and y is substituted by x.

Why it helps, and the cost. Sweeping finds equivalences and units that need case splits, which no propagation-based method finds, and it does so with search on problems of a few hundred variables instead of the whole formula. The cost is many small SAT calls, bounded by kitten "ticks" (`sweepeffort`) and by the environment limits. kissat runs it in every probe phase and, by default, during initial preprocessing (`preprocessweep`). kitten is reused elsewhere in kissat too: `definition.c` uses it to find gate definitions for variable elimination from an unsatisfiable core.

CaDiCaL: — (not in 2.1.3) · kissat: `sweep.c`, `kitten.c` (option `sweep`) · MiniSat: — · ksat port: not ported

Summary of this chapter

technique	derives	CaDiCaL 2.1.3	kissat 4.0.4
failed literal probing	units	`probe.cpp`	— (see backbone)
hyper-binary resolution	binary clauses	`probe.cpp`	—
vivification	shorter / deleted clauses	`vivify.cpp`	`vivify.c`

equivalent literal substitution	equivalences (from BIG cycles)	`decompose.cpp`	`substitute.c`
transitive reduction	deleted binary clauses, some units	`transred.cpp`	`transitive.c`
ternary resolution	binary and ternary clauses	`ternary.cpp`	—
backbone	units	—	`backbone.c` (binary clauses)
congruence closure	equivalences (from gates)	—	`congruence.c`
SAT sweeping	equivalences and units	—	`sweep.c`, `kitten.c`

MiniSat has none of these techniques, and none of them is ported in the ksat Kotlin ports.

Using a solver incrementally

Chapters 1–5 looked at one call: a formula goes in, SAT or UNSAT comes out. Programs that embed a solver rarely ask just one question. A puzzle helper asks "can this cell be a mine?" for every cell, a planner asks "is there a plan of length k?" for growing k, a model counter asks "is there another solution?" until the answer is no. The questions share almost all of their clauses. This chapter is about asking them on one solver instance that stays alive, and about the interfaces (IPASIR, IPASIR-UP) that let a program talk to the solver while it searches.

Incremental SAT and the IPASIR interface

The intuition: a solver that just finished a solve call has paid for a lot of knowledge. It has learned clauses (chapter 1), variable scores and saved phases (chapter 2), a cleaned-up clause database (chapter 3). All of that is still valid for the next question as long as the next question only adds constraints or makes temporary ones. Throwing the instance away and loading the formula again means paying for that knowledge again.

IPASIR is the small C interface the SAT competition's incremental track standardized, and every incremental solver understands some version of it. Its core is five calls:

IPASIR	ksat	meaning
`ipasir_add`	`addClause(lits)`	add a clause (IPASIR: one literal per call, 0 closes it)
`ipasir_assume`	`solve(assumptions)`	assume a literal for the next solve only
`ipasir_solve`	`solve(...)`	10 = SAT, 20 = UNSAT (0 = interrupted)
`ipasir_val`	`valueOf(v)`	value of a variable in the model, after SAT
`ipasir_failed`	`failed(lit)`	was this assumption part of the reason for UNSAT?

After a solve the program may add more clauses and solve again. Clauses stay forever; assumptions are forgotten after each solve. That split is the whole design: permanent facts go in as clauses, "what if" questions go in as assumptions.

> **Tie and shirt (the example from cadical.hpp)**
> Let a = "wear a tie", b = "wear a shirt", with $C_1 = (\overline{a} \vee b)$, $C_2 = (a \vee b)$, $C_3 = \left(\overline{a} \vee \overline{b}\right)$. From C_1 and C_2 the shirt is forced, and then C_3 forbids the tie: the only model is $a = 0, b = 1$.
>
> Now ask "what if I wear a tie?" by assuming a: UNSAT, and `failed(a)` says the tie is to blame. Then ask "what if no shirt?" by assuming $\overline{b}$: UNSAT again, and this time $\overline{b}$ is the failed assumption. The clauses were loaded once; the two questions left no trace in the formula.

```
val k = Ksat(Solver.CADICAL, numVars = 2)       // a = tie = 1, b = shirt = 2
k.addClause(intArrayOf(-1, 2))
k.addClause(intArrayOf(1, 2))
k.addClause(intArrayOf(-1, -2))
check(k.solve() == SatResult.SAT)
check(!k.valueOf(1) && k.valueOf(2))            // no tie, shirt

check(k.solve(assumptions = intArrayOf(1)) == SatResult.UNSAT)
check(k.failed(1))                               // the tie is to blame

check(k.solve(assumptions = intArrayOf(-2)) == SatResult.UNSAT)
check(k.failed(-2))                              // "no shirt" is to blame
```

Why it matters: learned clauses are consequences of the clauses alone, so they help every later question. On a sequence of similar queries the second and later solves are often much faster than the first. IPASIR also has `ipasir_set_terminate` (stop a long solve) and `ipasir_set_learn` (export learned clauses); ksat has neither.

CaDiCaL: `ipasir.h`, `ipasir.cpp`, the IPASIR block of `cadical.hpp` · kissat: — (kissat 4.0 refuses a second search: "incremental solving not supported") · MiniSat: `Solver.h` (`solve(assumps)`, `addClause`) · ksat: `SatSolver` (`addClause`, `solve(assumptions)`, `valueOf`, `failed`) behind the `Ksat` facade. kissat's incremental support is a port extension. MicroSat (ksat's minimal fourth solver, no incremental API) only does a plain single solve.

Assumptions are decisions on the first levels

The intuition: an assumption is not a clause. The solver treats the assumptions as its first decisions, one decision level per assumption, in the given order. Only when all of them are on the trail does it start choosing decisions of its own. Everything else in CDCL stays the same: unit propagation after each assumption, conflict analysis, backjumping.

Two cases need care. If an assumption is already true when its turn comes (it was propagated by an earlier one), the solver opens an empty "pseudo" decision level, so that level i still belongs to assumption i. If an assumption is already false, the formula plus the assumptions is UNSAT and the solver stops right there. That is "UNSAT under these assumptions", not UNSAT of the formula.

Assumptions on the trail

Take $C_1 = (\overline{a} \vee c)$, $C_2 = \left(\overline{b} \vee \overline{c} \vee d\right)$, $C_3 = \left(\overline{d} \vee e\right)$ and assume a, b. The trail is

$$a@1, \quad c@1 \leftarrow C_1, \quad b@2, \quad d@2 \leftarrow C_2, \quad e@2 \leftarrow C_3$$

Level 1 is the assumption a and its consequences, level 2 the assumption b and its consequences. Here all five variables are already assigned, so the solve returns SAT with $a = b = c = d = e = 1$ (the only model with a and b true).

- Assume a, c, b instead: when c's turn comes, c is already true from level 1. The solver opens an empty level 2 and puts b on level 3.
- Assume $a, b, \overline{e}$: after level 2, e is true, so the third assumption is already false. Result: UNSAT under these assumptions. Without assumptions the formula is of course SAT.

Why it matters: a clause learned while assumptions are on the trail is derived by resolution from the clauses only. The assumptions were decisions, and decisions never become part of a learned clause's justification, they only steer the search. So everything learned under assumptions a, b remains valid when the next call assumes $\overline{a}$. Had the program added the units (a) and (b) instead, they would stay forever and the next question would be answered for the wrong formula.

CaDiCaL: `decide.cpp` (`decide`: next assumption, pseudo level if already true, 20 if false), `assume.cpp` · kissat: — (no assumptions upstream) · MiniSat: `Solver.cc` (`search`: "Perform user provided assumption", dummy decision level) · ksat: `solve(assumptions)` for MiniSat, CaDiCaL and kissat (kissat: port extension), not MicroSat. After each `solve(assumptions)` the ports backtrack to level 0, so the instance is ready for the next call.

The failed-assumption core

The intuition: when the answer is UNSAT under assumptions, the program usually wants to know **which** assumptions caused it. "Can this cell be a mine, given these five clues?" is more useful with "no, because of clues 2 and 4". The solver can answer that cheaply from the trail it has at the moment it stops: the falsified assumption was falsified by unit propagation, and following the reasons backwards through the implication graph ends at decisions. All those decisions are assumptions (the solver had not made its own decisions yet), so they form the core.

Both solvers do this walk, with the same idea:

- MiniSat's `analyzeFinal` starts from the falsified assumption, walks the trail backwards and marks the reason literals of every seen variable. Seen variables without a reason are decisions, that is assumptions, and go into the core.
- CaDiCaL's `failing()` first looks for three cases: (1) an assumption falsified at the root (then that one assumption is the core), (2) two clashing assumptions x and $\overline{x}$ (both are the core), otherwise (3) the falsified assumption on the lowest decision level, from which it runs a BFS over the reasons, as MiniSat does.

A core that is not minimal

$C_1 = \left(\overline{a} \vee \overline{b} \vee c\right)$, $C_2 = (\overline{a} \vee d \vee c)$, $C_3 = \left(\overline{a} \vee \overline{d} \vee c\right)$, assumptions $a, b, \overline{c}$.

- Level 1: $a@1$. Nothing propagates: C_2 and C_3 each still have two open literals.
- Level 2: $b@2$, then $c@2 \leftarrow C_1$.
- Level 3: the assumption $\overline{c}$ is already false. UNSAT under the assumptions.

The walk starts at c. Its reason C_1 points at a and b; both have no reason, so both are assumptions. The core is $\{a, b, \overline{c}\}$: `failed` is true for all three. Both the MiniSat and the CaDiCaL port report exactly this.

The core is correct (those three assumptions alone are UNSAT) but not minimal. C_2 and C_3 resolve on d to $(\overline{a} \vee c)$, so a and $\overline{c}$ alone already clash. Unit propagation cannot see that without a case split on d, so the solver used the path via C_1 and blamed b as well. Assuming only $b, \overline{c}$ is SAT, so b really is innocent.

```
val k = Ksat(Solver.MINISAT, numVars = 4)         // a=1, b=2, c=3, d=4
k.addClause(intArrayOf(-1, -2, 3))                // C1
k.addClause(intArrayOf(-1, 4, 3))                 // C2
k.addClause(intArrayOf(-1, -4, 3))                // C3
val assumptions = intArrayOf(1, 2, -3)
check(k.solve(assumptions) == SatResult.UNSAT)
val core = assumptions.filter { k.failed(it) }    // [1, 2, -3]
```

The answer also depends on what the solver has learned. Solve once more under only $a, \overline{c}$: that is UNSAT too, and to find out the solver learns $(\overline{a} \vee c)$. Ask the first question again and the trail becomes $a@1, c@1$, so the core is now $\{a, \overline{c}\}$ (both ports).

Why it matters: cores are what make many applications work (explanations, MaxSAT, diagnosis, finding which constraints of a configuration conflict). `cadical.hpp` says it plainly: "the core does not have to be minimal". If the program needs a minimal core, it shrinks it itself: drop one assumption from the core,

solve under the rest, and keep it dropped if the answer is still UNSAT. Each step is a cheap incremental call. A practical detail: CaDiCaL only allows `failed` in the UNSAT state. After a SAT result, or once a clause is added, upstream CaDiCaL aborts with an API contract violation (`REQUIRE`), and the ksat port throws an `IllegalStateException`. MiniSat simply answers false.

CaDiCaL: `assume.cpp` (`failing`, `failed`) · kissat: — (no failed-assumption API upstream) · MiniSat: `Solver.cc` (`analyzeFinal`, the `conflict` vector) · ksat: `failed(lit)` for CaDiCaL and MiniSat. The CaDiCaL port computes the core at the UNSAT exit, before its backtrack to level 0 (upstream computes it lazily).

Adding clauses after a solve

The intuition: after a SAT answer, the trail is full. Every variable is assigned, many on high decision levels. A new clause may already be falsified by that trail, or be unit, or be satisfied by a decision that will be undone soon. None of this says anything about the new clause in general; it only reflects the last model. So before a new original clause goes in, the solver goes back to the root (level 0). There the clause is checked against the root assignment only: root-false literals are dropped, a root-satisfied clause is ignored, and a clause that becomes unit is assigned at level 0, where it belongs.

A unit after a solve

$F = (a \vee b)$. The first solve may return $a = 1, b = 1$ (both decisions: $a@1, b@2$). Now the program learns that a must be false and adds $(\overline{a})$.

On the old trail, $(\overline{a})$ would be falsified at level 1, and a unit at level 1 is undone by the next backtrack. Going back to level 0 first gives the right picture: $\overline{a}@0$, then $b@0 \leftarrow (a \vee b)$. The next solve returns $a = 0, b = 1$, the only model left.

```
val k = Ksat(Solver.CADICAL, numVars = 2)
k.addClause(intArrayOf(1, 2))
k.solve()                          // SAT, e.g. a = 1, b = 1
k.addClause(intArrayOf(-1))        // new fact: a is false
k.solve()                          // SAT, a = 0, b = 1
```

Why it matters: this is the “add” half of incremental solving, and it is easy to get subtly wrong. The CaDiCaL and kissat ports once got exactly this example wrong: the unit landed on a dirty level above 0, so the next backtrack removed it again (CaDiCaL) or the decision queue walk crashed (kissat). Upstream avoids it in the same simple way: CaDiCaL’s `add_new_original_clause` starts with `if (!from_propagator && level && !opts.ilb) backtrack ();`, so it backtracks to level 0 first (unless the `ilb` option is set, default off). MiniSat’s `addClause_` asserts decision level 0 and `solve_` ends with `cancelUntil(0)`.

CaDiCaL: `clause.cpp` (`add_new_original_clause`) · kissat: — (no second solve upstream; the port goes back to level 0 before importing a clause) · MiniSat: `Solver.cc` (`addClause_`, `solve_`) · ksat: `addClause` after `solve` for MiniSat, CaDiCaL and kissat. Not MicroSat.

Model enumeration with blocking clauses

The intuition: to list all models, find one, forbid exactly that one, and ask again. The clause that forbids a model M is the OR of the negations of M’s literals; it is false under M and true under every other assignment. When the answer becomes UNSAT, every model has been seen once. With an incremental solver each round is one `addClause` plus one `solve`, and learned clauses carry over.

Enumerating XOR

$a \oplus b$ in CNF is $(a \vee b) \wedge (\overline{a} \vee \overline{b})$, with the two models $\{a = 0, b = 1\}$ and $\{a = 1, b = 0\}$.

1. Solve: $a = 0, b = 1$. Block it with $(a \vee \overline{b})$.
2. Solve: $a = 1, b = 0$. Block it with $(\overline{a} \vee b)$.
3. Solve: UNSAT. The four clauses rule out all four assignments. Done, two models.

```
val xor = listOf(intArrayOf(1, 2), intArrayOf(-1, -2))
val models = enumerateModels(Solver.CADICAL, numVars = 2, cnf = xor,
                             incremental = true).toList()
// [[false, true], [true, false]]
```

Each model is a full assignment over the variables $1, ..., n$, with n = `numVars`. A variable no clause constrains doubles the count: $(a \vee b)$ with `numVars = 3` has $3 \cdot 2 = 6$ models. That is correct, but it surprises people. If only some variables matter (a projection), block only their literals; then one blocking clause removes a whole group of models. `enumerateModels` does not do that, it always blocks all of $1, ..., n$.

Why it matters: blocking clauses are the simplest all-SAT method and need nothing but add-after-solve. They get slow for many models: every model adds a clause of length n. For counting large model sets, dedicated model counters are the better tool.

CaDiCaL / kissat / MiniSat: — (a technique on top of the API, not inside the solver) · ksat: `Enumerate.kt` (`enumerateModels`). `incremental = false` (default) builds a fresh solver per round and works with every solver; `incremental = true` keeps one solver and needs add-after-solve (MiniSat, CaDiCaL, kissat; throws for MicroSat).

Frozen variables and elimination

The intuition: chapter 4 showed bounded variable elimination: a variable x is removed by replacing its clauses with all their resolvents on x. That keeps the formula **equisatisfiable**, not equivalent. It is safe for a single solve. It breaks incremental use if the program later mentions x again, in a new clause or an assumption, because the solver no longer has the clauses that constrained x.

Chapter 4 (section “Frozen variables”, example “A spurious model after elimination”) shows the failure: after x is eliminated, a later call that assumes $x = 0$ and $a = 0$ gets a model, although the original clauses make that combination UNSAT. The same happens when the program adds new clauses $(\overline{x})$ and $(\overline{a})$ instead of assuming them: the solver no longer has $(x \vee a)$, so nothing stops it.

There are two fixes. The old one is **freezing**: the program declares up front which variables it will use again (MiniSat’s `setFrozen`, CaDiCaL’s `freeze`/`melt`), and frozen variables are never eliminated. MiniSat’s `SimpSolver` also freezes the assumption variables during a solve. The newer one is restoring: CaDiCaL notices when an eliminated variable shows up again and puts the removed clauses back from the extension stack, so the program does not need to freeze anything. Variables observed by an IPASIR-UP propagator (below) are frozen internally.

Why it matters: preprocessing and incremental use pull in opposite directions. Elimination wants to remove variables, the program wants to keep talking about them. Freezing, restoring or not eliminating at all are the three ways out.

CaDiCaL: `cadical.hpp` (`freeze`, `melt`, `frozen`), `restore.cpp`, `elim.cpp` · kissat: — (single solve) · MiniSat: `simp/SimpSolver.h` and `SimpSolver.cc` (`setFrozen`, `extra_frozen`) · ksat: no port has elimination (MiniSat is ported without `SimpSolver`, CaDiCaL without `elim`), so there is nothing to freeze and no freeze API.

IPASIR-UP: user propagators

The intuition: some constraints are awkward as clauses. They need too many clauses (an at-most-one over thousands of variables, a graph property, an arithmetic check), or the program only knows them as code. IPASIR-UP lets the program take part in the search instead. It plugs a **propagator** object into the solver, tells it which variables it cares about (the observed variables), and from then on the solver calls the propagator during CDCL:

callback (CaDiCaL name)	**what it is for**
`notify_assignment`, `notify_new_decision_level`, `notify_backtrack`	the solver reports assignments to observed variables and level changes, so the propagator can keep its own copy of the trail. Not necessarily eager: usually delivered right before the next question to the propagator.
`cb_propagate`	“do you imply a literal under the current assignment?” Returns a literal or 0.
`cb_add_reason_clause_lit`	the reason clause for a literal returned by `cb_propagate`, one literal per call, closed by 0. It must contain the propagated literal. Asked only when the solver needs it.
`cb_has_external_clause`, `cb_add_external_clause_lit`	hand the solver a new clause during search, one literal per call. Added as an irredundant clause unless marked forgettable.
`cb_check_found_model`	the solver has a complete model. Return false to reject it; the propagator must then give a clause (or observe new variables) that explains why.
`cb_decide`	suggest the next decision literal, or 0 to let the solver choose.

Two ideas make this efficient. First, **lazy reasons**: `cb_propagate` only says “this literal is implied”. The solver puts it on the trail with a placeholder reason and asks for the actual clause later. With the default option `exteagerreasons`, CaDiCaL asks, before each conflict analysis, for the reasons of all external propagations the conflict depends on. A literal propagated at level 0 has its reason asked for right away. Many propagated literals are undone before anybody asks. Second, every clause the propagator ever hands over is a normal clause from then on. The solver propagates it itself and learns from it, so the propagator is not asked the same thing twice.

The propagator must be honest: every literal it propagates must follow from its reason clause under the current assignment, and every clause it adds must be implied by its constraint. Otherwise

the solver's answers are wrong. In Kotlin the callbacks have camelCase names (`cbPropagate`, `notifyAssignment`, …), and `Ksat` hosts the propagator (`connectExternalPropagator`, `addObservedVar`, `isDecision`, `forceBacktrack`).

A lazy at-most-one, step by step

The solver gets $C_1 = (a \vee b \vee c)$, $C_2 = (b \vee d)$, $C_3 = \left(b \vee \overline{d}\right)$. The propagator knows the constraint "at most one of a, b, c", which as clauses would be $\left(\overline{a} \vee \overline{b}\right)$, $(\overline{a} \vee \overline{c})$, $\left(\overline{b} \vee \overline{c}\right)$. The solver never sees those three clauses up front. Observed variables: a, b, c. The models of the whole problem are $a = 0, b = 1, c = 0$ with d free.

This is the real callback sequence of the CaDiCaL port, with `cb_decide` suggesting a first so that the run is reproducible:

1. `cb_decide` returns a. Trail $a@1$. The propagator gets `notify_new_decision_level` and `notify_assignment([a])`.
2. `cb_propagate`: a is true, so b must be false. It returns $\overline{b}$ and only remembers the reason $\left(\overline{a} \vee \overline{b}\right)$. Trail $\overline{b}@1 \leftarrow \text{ext}$.
3. The solver propagates its own clauses: $d@1 \leftarrow C_2$, and then $C_3 = \left(b \vee \overline{d}\right)$ is false. Conflict.
4. The conflict depends on the external propagation $\overline{b}$. With the default `exteagerreasons`, CaDiCaL asks for the reasons of all external propagations the conflict depends on, here $\overline{b}$. It calls `cb_add_reason_clause_lit` with the literal $\overline{b}$ three times and gets $\overline{a}, \overline{b}, 0$.
5. Resolving C_3 with C_2 on d gives the learned unit (b). Backtrack to level 0 (`notify_backtrack(0)`), $b@0$.
6. The solver now owns $\left(\overline{a} \vee \overline{b}\right)$, so it propagates $\overline{a}@0$ by itself. The propagator is only notified, it is never asked for $\overline{a}$.
7. `cb_propagate` returns $\overline{c}$. At level 0 the solver asks for the reason immediately and gets $\left(\overline{b} \vee \overline{c}\right)$.
8. `cb_propagate` returns 0. The solver decides d (not observed, so the notification is empty), the model is complete, and `cb_check_found_model([-a, b, -c])` accepts it. SAT.

Without the `cb_decide` hint, the port's first decision level already makes b true. Then $\overline{a}$ and $\overline{c}$ are propagated, nothing conflicts, and `cb_add_reason_clause_lit` is never called at all: those reasons were never needed.

If the propagator never propagated (`cb_propagate` always 0), it would only see complete models: the run gets a, b, c all true, rejects that model and adds $\left(\overline{a} \vee \overline{b}\right)$, later rejects $\overline{a}, b, c$ and adds $\left(\overline{b} \vee \overline{c}\right)$, and then accepts $\overline{a}, b, \overline{c}$. Same answer, more search.

The at-most-one propagator in Kotlin

```
import org.bytefred.ksat.ExternalPropagator
import org.bytefred.ksat.OutBoolean
import org.bytefred.ksat.SatResult
import org.bytefred.ksat.facade.Ksat
import org.bytefred.ksat.facade.Solver
import kotlin.math.abs

/** At most one of [vars] is true. Eager propagation, lazy reasons. */
class AtMostOne(private val vars: IntArray, firstDecision: Int = 0) :
    ExternalPropagator {
    private val trail = mutableListOf(mutableListOf<Int>()) // lits per level
    private val value = HashMap<Int, Int>()          // var -> assigned literal
    private val reasons = HashMap<Int, IntArray>()   // propagated lit -> reason
    private var pending: ArrayDeque<Int>? = null    // reason being handed out
    private val toAdd = ArrayDeque<IntArray>()       // clauses for the solver
```

```
    private var addPos = 0
    private var suggestion = firstDecision

    override fun notifyAssignment(lits: IntArray) {
        for (lit in lits) { value[abs(lit)] = lit; trail.last().add(lit) }
    }

    override fun notifyNewDecisionLevel() { trail.add(mutableListOf()) }

    override fun notifyBacktrack(newLevel: Int) {
        while (trail.size - 1 > newLevel)
            for (lit in trail.removeAt(trail.lastIndex)) value.remove(abs(lit))
    }

    override fun cbDecide(): Int = suggestion.also { suggestion = 0 }

    override fun cbPropagate(): Int {
        val t = vars.firstOrNull { value[it] == it } ?: return 0 // a true var?
        val u = vars.firstOrNull { it != t && value[it] != -it } ?: return 0
        reasons[-u] = intArrayOf(-t, -u)        // keep the reason, do not send
        return -u
    }

    override fun cbAddReasonClauseLit(propagatedLit: Int): Int {
        val q = pending ?: ArrayDeque(reasons.getValue(propagatedLit).toList())
        pending = q
        if (q.isEmpty()) { pending = null; return 0 }
        return q.removeFirst()
    }

    override fun cbCheckFoundModel(model: IntArray): Boolean {
        val trueVars = model.filter { it > 0 && it in vars }
        if (trueVars.size <= 1) return true
        toAdd.addLast(intArrayOf(-trueVars[0], -trueVars[1])) // reject, explain
        return false
    }

    override fun cbHasExternalClause(isForgettable: OutBoolean) =
        toAdd.isNotEmpty()

    override fun cbAddExternalClauseLit(): Int {
        val c = toAdd.first()
        if (addPos < c.size) return c[addPos++]
        toAdd.removeFirst(); addPos = 0
        return 0
    }
}

val k = Ksat(Solver.CADICAL, numVars = 4)     // a=1, b=2, c=3, d=4
k.addClause(intArrayOf(1, 2, 3))              // C1
k.addClause(intArrayOf(2, 4))                 // C2
k.addClause(intArrayOf(2, -4))                // C3
k.connectExternalPropagator(AtMostOne(intArrayOf(1, 2, 3), firstDecision = 1))
for (v in 1..3) k.addObservedVar(v)
check(k.solve() == SatResult.SAT)             // a = 0, b = 1, c = 0
```

If two variables are true at once, `cbPropagate` returns the negation of a true literal. The solver then gets a reason clause that is false under the trail and treats it as a conflict. `cbCheckFoundModel` is the safety net: even if a propagation was missed, a bad model is rejected with a clause.

A few more pieces of the interface. `is_lazy` (Kotlin: `isLazy`) declares a propagator that only checks complete models and is never asked to propagate. `are_reasons_forgettable` lets the solver delete reason clauses again during clause database reduction. `is_decision(lit)` tells whether an observed literal was a decision. `force_backtrack(level)`, only effective inside `cb_decide`, makes the solver jump back. Observed variables can be added during the search, but removed only between solves. And API calls from inside a callback (adding a clause, solving) are not allowed: upstream CaDiCaL aborts with an API contract violation (`REQUIRE`), the ksat port throws an exception.

Why it matters: IPASIR-UP turns the SAT solver into the search engine of a bigger system: SMT-style theory reasoning, symmetry breaking, graph constraints, or simply constraints too large to write down. The lazy design keeps the price low: the propagator explains only the inferences that end up in a conflict.

CaDiCaL: `cadical.hpp` (`ExternalPropagator`, IPASIR-UP block of `Solver`), `external_propagate.cpp` (`external_propagate`, `external_check_solution`, `ask_decision`, `explain_external_propagations`, which `analyze.cpp` calls on conflicts when option `exteagerreasons` is set) · kissat: — · MiniSat: — · ksat: CaDiCaL only; `ExternalPropagator.kt` in ksat-common, `Ksat.connectExternalPropagator`, `addObservedVar`, `isDecision`, `forceBacktrack`. Other solvers throw `UnsupportedOperationException`.

Forced phases and the fixed-assignment listener

Two small CaDiCaL extras that are not IPASIR-UP but help a program steer and watch the solver.

Forced phases. Chapter 2 explained phase saving: when the solver decides a variable, it uses the value the variable had last time. `phase(lit)` overrides that: whenever the solver decides this variable, it takes the sign of `lit`. `unphase` removes the override. It is a preference, not a constraint. If propagation already assigned the variable, the phase is never consulted.

Fixed-assignment listener. A variable is fixed when it is assigned at level 0, that is, it is implied by the formula and will never change again (until the end of the instance). A connected listener gets `notify_fixed_assignment(lit)` for every newly fixed literal: original units, their root propagation, learned units. Listeners are real-time: literals fixed before connecting are not replayed.

A phase is only a preference

$F = \left(\overline{a} \vee \overline{b}\right)$ with `phase(a)` and `phase(b)`: the program would like both to be true, which F forbids. The solver decides one of them with its forced phase, and the other one is then propagated false. In the CaDiCaL port the answer is $a = 0, b = 1$ (b is decided first). With `phase(-a)` and `phase(b)` the wish is a model, and the port returns exactly it.

```
val k = Ksat(Solver.CADICAL, numVars = 2)
k.addClause(intArrayOf(-1, -2))
k.phase(1); k.phase(2)
k.solve()                          // SAT, a = 0, b = 1
```

Watching variables become fixed

$C_1 = (a \vee b)$, $C_2 = \left(\overline{b} \vee \overline{c}\right)$, $C_3 = (c \vee d)$, then the unit $(\overline{a})$. The unit fixes $\overline{a}@0$, and root propagation fixes $b@0 \leftarrow C_1$, $\overline{c}@0 \leftarrow C_2$, $d@0 \leftarrow C_3$. The listener hears $-1, 2, -3, 4$, already while the unit is added (CaDiCaL propagates new units right away).

```
val k = Ksat(Solver.CADICAL, numVars = 4)
k.connectFixedListener(object : FixedAssignmentListener {
    override fun notifyFixedAssignment(lit: Int) = println("fixed $lit")
})
k.addClause(intArrayOf(1, 2))
k.addClause(intArrayOf(-2, -3))
k.addClause(intArrayOf(3, 4))
k.addClause(intArrayOf(-1))      // prints: fixed -1, fixed 2, fixed -3, fixed 4
```

Why it matters: forced phases let a program say "I expect roughly this solution" (for example the previous solution in a sequence of similar queries) and get it back when it is still a model. In the ksat port that holds exactly, because the port has no lucky phases or local search; upstream CaDiCaL may try those first and return another model. The fixed listener lets the program learn facts that hold in every model while the solver runs, for example to prune its own search space or to stop asking questions whose answer is already fixed.

CaDiCaL: `decide.cpp` (`decide_phase` reads `phases.forced` first), `phases.cpp` (`phase`, `unphase`), `cadical.hpp` (`FixedAssignmentListener`), `flags.cpp` (`mark_fixed`) · kissat: — · MiniSat: — (only `setPolarity`, not exposed in ksat) · ksat: CaDiCaL only; `ForcedPhases` and `FixedListening` in ksat-common, `Ksat.phase`, `unphase`, `connectFixedListener`, `disconnectFixedListener`.

Proofs of unsatisfiability: DRAT and LRAT

The intuition: a SAT answer comes with a model that anybody can check in linear time. An UNSAT answer has nothing like that, unless the solver writes down how it got there. A clausal proof is the list of clauses the solver learned, in order, ending with the empty clause. A checker verifies that each clause follows from the earlier ones by unit propagation alone (RUP: assume the clause false, propagate, get a conflict).

- **DRAT** lists the learned clauses (and deleted ones, marked `d`). Each added clause must be RUP or RAT. RAT ("resolution asymmetric tautology") also admits a clause that is blocked-like on its first literal: every resolvent on that literal is RUP. That is what justifies the steps of chapter 4 that are not plain consequences, such as BVE and BCE. The checker has to find the propagation for each clause itself, which can take longer than solving.
- **LRAT** adds clause IDs and, for each clause, the IDs of the clauses that become unit in its propagation (the hints). The checker only replays them, which is simple and fast enough to be done by a formally verified checker.

A two-step proof

$C_1 = (a \vee b)$, $C_2 = \left(a \vee \overline{b}\right)$, $C_3 = (\overline{a} \vee c)$, $C_4 = (\overline{a} \vee \overline{c})$ is UNSAT.

- Clause 5 = (a): assume $\overline{a}$; C_1 gives b, then C_2 is false.
- Clause 6 = empty: clause 5 gives a, C_3 gives c, then C_4 is false.

As DRAT that is the two lines `1 0` and `0`. As LRAT it is `5 1 0 1 2 0` and `6 0 5 3 4 0` (ID, literals, 0, hint IDs in propagation order, 0).

Why it matters: solvers have bugs, and proofs make UNSAT answers trustworthy without trusting the solver. In the incremental setting it gets harder: assumptions, clauses added later and propagator clauses all need a place in the proof, which is why CaDiCaL has incremental proof formats as well. None of this is ported to ksat. The ports were checked differently, by shadowing the C solvers trace step by step (see `shadowing-methodology`).

CaDiCaL: `proof.cpp`, `drattracer.cpp`, `lrattracer.cpp` (plus FRAT, IDRUP and LIDRUP tracers), `cadical.hpp` (`trace_proof`) · kissat: `proof.c` (DRAT, text or binary) · MiniSat: — (2.2.0 writes no proofs) · ksat: not ported in any solver.

Overview: who has what

One page to look things up. “Upstream” columns say whether the original solver has the technique (checked against the source trees: CaDiCaL 2.1.3, kissat 4.0.x, MiniSat 2.2.0, with the file that implements it). The last column says whether the ksat Kotlin ports have it. “core” means MiniSat’s `core/Solver.cc`, “simp” its optional `simp/SimpSolver.cc` layer. “all three” in the last column means the MiniSat, CaDiCaL and kissat ports (MicroSAT has none of these techniques beyond plain CDCL).

The CDCL loop and heuristics (chapters 1–3)

Technique	MiniSat	CaDiCaL	kissat	ksat ports
Unit propagation, watched literals	✓ core	✓ `propagate.cpp`	✓ `propsearch.c`, `proplit.h`	all three
1-UIP conflict analysis	✓ core	✓ `analyze.cpp`	✓ `analyze.c`	all three
Recursive minimization	✓ core	✓ `minimize.cpp`	✓ `minimize.c`	all three
Shrinking	–	✓ `shrink.cpp`	✓ `shrink.c`	not ported
Chronological backtracking	–	✓ (allowed by default)	✓ (allowed by default)	not ported
On-the-fly strengthening (OTFS)	–	✓ `analyze.cpp`	✓ `deduce.c`, `strengthen.c`	not ported
VSIDS / EVSIDS	✓ core	✓ `score.cpp`	✓ heap	all three
VMTF queue	–	✓ `queue.cpp`	✓ queue	CaDiCaL, kissat
Phase saving	✓ core	✓ `phases.cpp`	✓ `phases.c`	all three
Target / best phases	–	✓	✓	CaDiCaL, kissat
Stable / focused modes	–	✓ `restart.cpp`	✓ `mode.c`	CaDiCaL, kissat
Luby / reluctant restarts	✓ core	✓	✓ `restart.c`	all three
Glucose-style EMA restarts	–	✓ `ema.cpp`	✓	CaDiCaL, kissat
Trail reuse on restart	–	✓	✓	kissat
Rephasing	–	✓ `rephase.cpp`	✓ `rephase.c`	not ported
Reorder	–	–	✓ `reorder.c`	not ported

Warmup	–	–	✓ `warmup.c`	not ported
Lucky phases	–	✓ `lucky.cpp`	✓ `lucky.c`	not ported
Local search (walk)	–	✓ `walk.cpp`	✓ `walk.c`	not ported
Lookahead / cubing	–	✓ `lookahead.cpp`	–	not ported
LBD / glue tiers + reduce	activity-based	✓ `reduce.cpp`	✓ `reduce.c`, `tiers.c`	all three (own variant each)
Garbage collection	✓ core	✓ `collect.cpp`	✓ `collect.c`	own memory layout
Variable compaction	–	✓ `compact.cpp`	✓ `compact.c`	not ported

Simplifying the formula (chapters 4–5)

Technique	MiniSat	CaDiCaL	kissat	ksat ports
Subsumption, strengthening	✓ simp	✓ `subsume.cpp`	✓ `forward.c`	not ported
Bounded variable elimination	✓ simp	✓ `elim.cpp`	✓ `eliminate.c`	not ported
Fast elimination (fastel)	–	–	✓ `fastel.c` (off by default, preprocessing only)	not ported
Gate / definition detection	–	✓ `gates.cpp`	✓ `gates.c`, `ands.c`, `ifthenelse.c`, `definition.c`, `equivalences.c`	not ported
Blocked clause elimination	–	✓ `block.cpp` (off by default)	–	not ported
Covered clause elimination	–	✓ `cover.cpp` (off by default)	–	not ported
Conditioning (globally blocked)	–	✓ `condition.cpp` (off by default)	–	not ported
Instantiation	–	✓ `instantiate.cpp` (off by default)	–	not ported
Factorization (BVA)	–	–	✓ `factor.c`	not ported
Model reconstruction	✓ simp	✓ `extend.cpp`	✓ `extend.c`	not needed (nothing eliminated)
Failed literal probing (+ hyper-binary resolution)	–	✓ `probe.cpp`	binary only, `backbone.c` (see below)	not ported
Vivification	–	✓ `vivify.cpp`	✓ `vivify.c`	not ported
Equivalent literal substitution	–	✓ `decompose.cpp`	✓ `substitute.c`	not ported
Transitive reduction	–	✓ `transred.cpp`	✓ `transitive.c`	not ported
Ternary resolution	–	✓ `ternary.cpp`	–	not ported

Backbone	–	–	✓ `backbone.c`	not ported
Congruence closure	–	–	✓ `congruence.c`	not ported
SAT sweeping (with kitten)	–	–	✓ `sweep.c`, `kitten.c`	not ported

kissat's `probe.c` only schedules other passes (congruence, substitute, vivify, sweep, …); failed literals over binary clauses are found by `backbone.c`, and kissat has no hyper-binary resolution.

Incremental use (chapter 6)

Feature	MiniSat	CaDiCaL	kissat	ksat ports
Assumptions	✓ core	✓ `assume.cpp`	–	MiniSat, CaDiCaL, kissat (kissat: port extension)
Add clauses after a solve	✓	✓	–	MiniSat, CaDiCaL, kissat
Failed-assumption core	✓ `analyzeFinal`	✓ `failing()`	–	MiniSat, CaDiCaL
IPASIR-UP user propagators	–	✓ `external_propagate.cpp`	–	CaDiCaL
Forced phases (phase/unphase)	polarity only	✓ `phases.cpp`	–	CaDiCaL
Fixed-assignment listener	–	✓	–	CaDiCaL
Proofs (DRAT / LRAT)	–	✓ `proof.cpp`, …	✓ `proof.c`	not ported

Why the ports stop where they do
The ksat ports translate the search core line by line and check it step by step against instrumented C references. Every simplification technique changes the formula and the search trace, so each one would need its counterpart in the C reference before it could be verified the same way. The core plus the incremental API is what the Kotlin side needs; the rest is where the original solvers get their speed on hard instances.